



1ACC0221 – BIG DATA

Sección: 18516

Program: Computer Science

Instructor: Carlos Adrian Alarcon Delgado

Team Members:

Code	Full Name
u20211d574	Fabiana Nayeli Mallma Villanueva
u202323591	Luis Isaac Lazaro Machado

Table of Contents

1. Project Proposal:	3
2. Data Sources and Access:	4
3. Dataset Design and Construction:	5
4. Diccionario de datos:	7
5. Exploratory Analysis and Dataset Scale:	8
6. One processed dataset V1:	9
7. EDA Analysis:	10
8. Feature Matrix Construction:	13
9. Dimensionality Reduction:	14
10. Comparison Table:	16
11. Technical interpretation of results:	17
12. Bibliography:	18

1. Project Proposal:

This project is developed within the ecosystem of open-source scientific artificial intelligence tools. This domain was selected due to the accelerated and continuous growth of public repositories related to models, libraries, frameworks, and research-oriented tools. According to GitHub's Octoverse 2024 report, during 2024 developers created more than 70,000 new public and open-source generative AI projects, while total contributions to this type of project increased by nearly 60%. This growth reflects a highly dynamic, broad, and increasingly difficult environment to explore in a structured way.

In this context, the main problem lies in the fact that the AI software ecosystem is growing in an explosive and disorganized manner. Every week, new GitHub repositories appear related to scientific papers, experimental implementations, or applied tools, but there is no structured system that allows users to discover, compare, and understand which repositories are truly influential beyond superficial metrics such as stars or forks. This creates difficulties in identifying similar repositories, recognizing thematic families, and prioritizing relevant tools within an increasingly large universe.

Based on this, the proposed system seeks to answer four specific questions:

- Which AI repositories exist within a specific domain, and which of them are similar to each other?
- To which thematic family or cluster does a given repository belong?
- What other tools should someone explore if they are already using a specific tool?
- Which repositories are structurally the most influential within the ecosystem, beyond their apparent popularity?

The dataset is suitable for the course because it combines free text in descriptions and topics, numerical variables such as stars and forks, temporal variables such as age and recent activity, and implicit relationships between repositories. This combination supports the subsequent stages of the project, including representation, clustering, recommendation, and graph analysis.

2. Data Sources and Access:

2.1. Source Inventory:

Source 1: GitHub API

URL: <https://api.github.com>

License: Public API under GitHub Terms of Service.

Format: JSON via REST API, processed with PyGithub 2.x.

Current size: 86 repositories and 13 columns, with projected growth to 500–1000 repositories in later iterations.

Source 2: Papers With Code

URL: <https://paperswithcode.com/api/v1>

License: Public API with no authentication required.

Format: JSON via REST API.

Intended use: To enrich each repository with its associated scientific paper, the task it addresses, and the reported benchmarks.

Source 3: Hugging Face Hub

URL: <https://huggingface.co/>

License: Public API with official Python SDK support.

Format: JSON.

Intended use: A tertiary source to enrich the catalog with models and datasets linked to the analyzed repositories.

2.2. Ethics and Access Note:

The data used in this project comes from public sources accessible through official APIs. At this stage, the main source is the GitHub API, which is accessed using a personal token only for authentication and rate-limit control.

No scraping of private profiles was performed, and no information restricted by third-party authentication was accessed. The extraction is limited to technical metadata from public repositories, such as descriptions, topics, primary language, stars, forks, and activity dates.

No real names, email addresses, private messages, or sensitive content from individual users were collected. Therefore, the risk related to personal data is minimal. In addition, the raw dataset will not be redistributed outside the team, and the processed artifacts will be used exclusively for academic purposes within the course.

3. Dataset Design and Construction:

3.1. Schema Draft:

Column	Type	Description	Key
repo_id	integer	Unique identifier of the repository on GitHub	PK
nombre	string	Full name in user/repository format	FK : papers, dependencies
descripcion	string	Short description provided by the author	
lenguaje	string	Main programming language	
stars	integer	Number of accumulated stars	
forks	integer	Number of repository forks	
topics	list	Thematic tags declared by the author	
fecha_creacion	datetime	Repository creation date	
ultimo_commit	datetime	Date of the last recorded commit	

3.2. Relaciones y joins esperados:

Table A	Table B	Join Field	Description
repos	papers	nombre	Links each repository with its related scientific paper.
repos	dependencies	repo_id	Links each repository with its declared libraries or dependencies.

repos	topics_expanded	repo_id	Normalizes topics into an expanded table for further analysis.
-------	-----------------	---------	--

4. Diccionario de datos:

Column	Type	Description	Range or Observed Values
repo_id	integer	Unique GitHub ID	7 digits or more
nombre	string	user/repo	Example: microsoft/qlib
descripcion	string	Free text provided by the author	1 to ~200 characters, “No description” if empty
lenguaje	string	Main language	Python, Jupyter Notebook, C++, TeX, Unknown
stars	integer	Superficial popularity	51 ,41,080 in the current dataset
forks	integer	Number of times it was forked	4 ,8,394
topics	list	Author-defined tags	Between 2 and 20 per repository
fecha_creacion	datetime	Creation date	Repositories with up to 3,714 days of age
ultimo_commit	datetime	Last recorded push	From today up to 3,055 days ago
url	string	Direct link	Always https://github.com/...
dias_sin_commit	integer	Recent inactivity	0 (active today) ,3,055 days
antiguedad_dias	integer	Repository age	402 ,3,714 days
num_topics	integer	Tag richness	2 ,20

5. Exploratory Analysis and Dataset Scale:

The key figures of the current dataset are as follows:

Total repositories: 86

Total columns: 13

Missingness after cleaning: 0 across all columns

Memory estimate: 38.4 KB

Stars range: 51 to 41,080, with a median of 1,241

This distribution is strongly right-skewed, which is typical in GitHub repositories where a small number of projects accumulate most of the stars.

Unique languages: 7

Python is the dominant language with 38 repositories, followed by 29 repositories with no declared language (documentation or notebooks not classified by GitHub).

Repositories active in the last 30 days: 16 out of 86

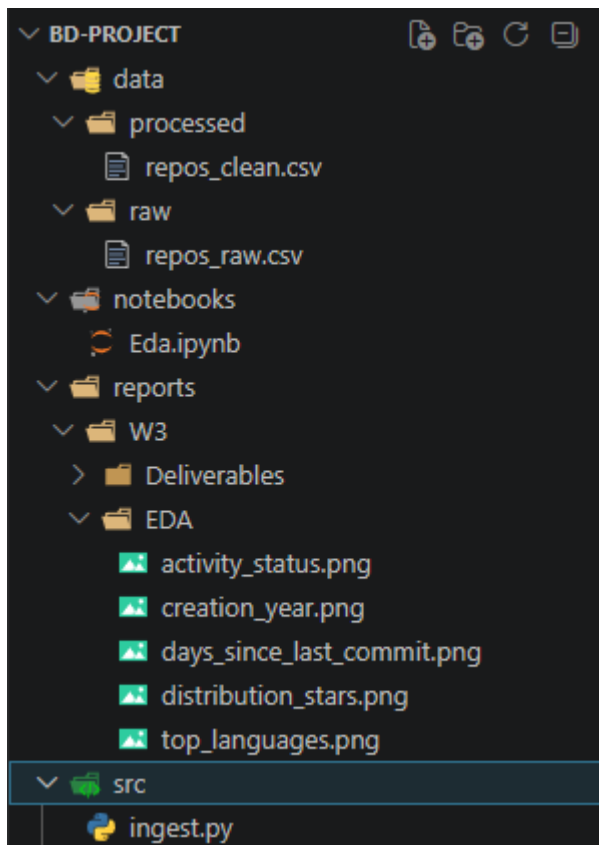
This suggests that most of the repositories correspond to research projects that are published but do not receive continuous maintenance.

Average age: 2,245 days (approximately 6 years)

This indicates that the dataset captures established tools within the ecosystem.

The chart [reports/scale_analysis.png](#) shows the log-scale distribution of stars, the ranking of programming languages, and the distribution of days since the last commit.

6. One processed dataset V1:



The first processed version of the dataset is generated through a pipeline organized in stages.

Step 1: the GitHub API is queried to collect the metadata of the repositories defined in the project queries.

Step 2: the result of this extraction is stored as a raw dataset in the path `data/raw/repos_raw.csv`.

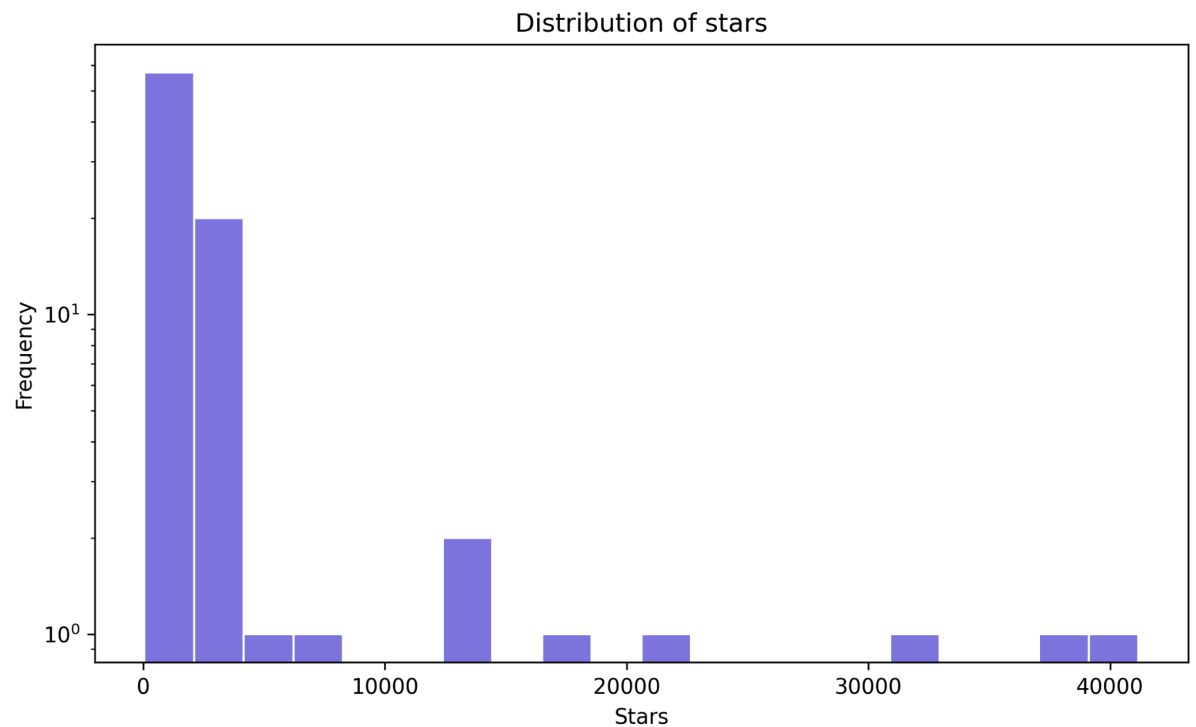
Step 3: this raw file is then subjected to a cleaning process, which includes handling missing values, normalizing variables, and creating new features such as `dias_sin_commit`, `antiguedad_dias` and `num_topics`.

Step 4: Finally, the result is saved as a processed dataset in the path `data/processed/repos_clean.csv`

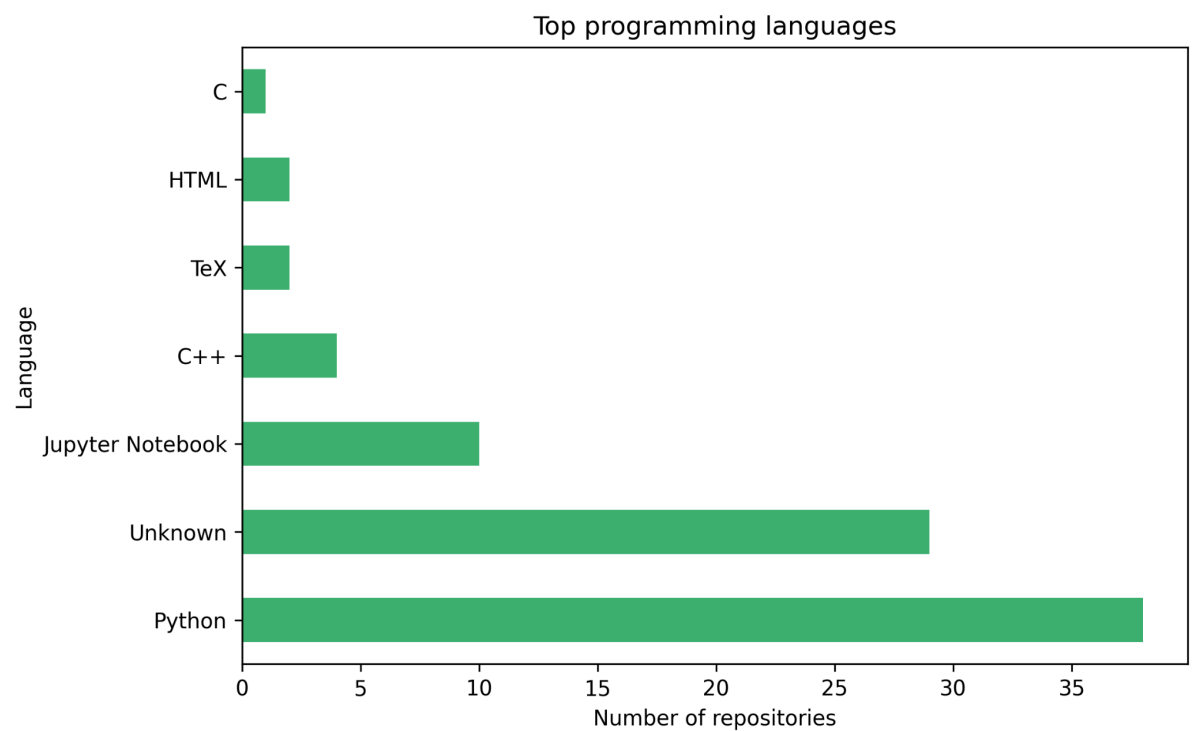
El pipeline puede ejecutarse desde la raíz del proyecto con el siguiente comando:

```
python src/ingest.py
```

7. EDA Analysis:

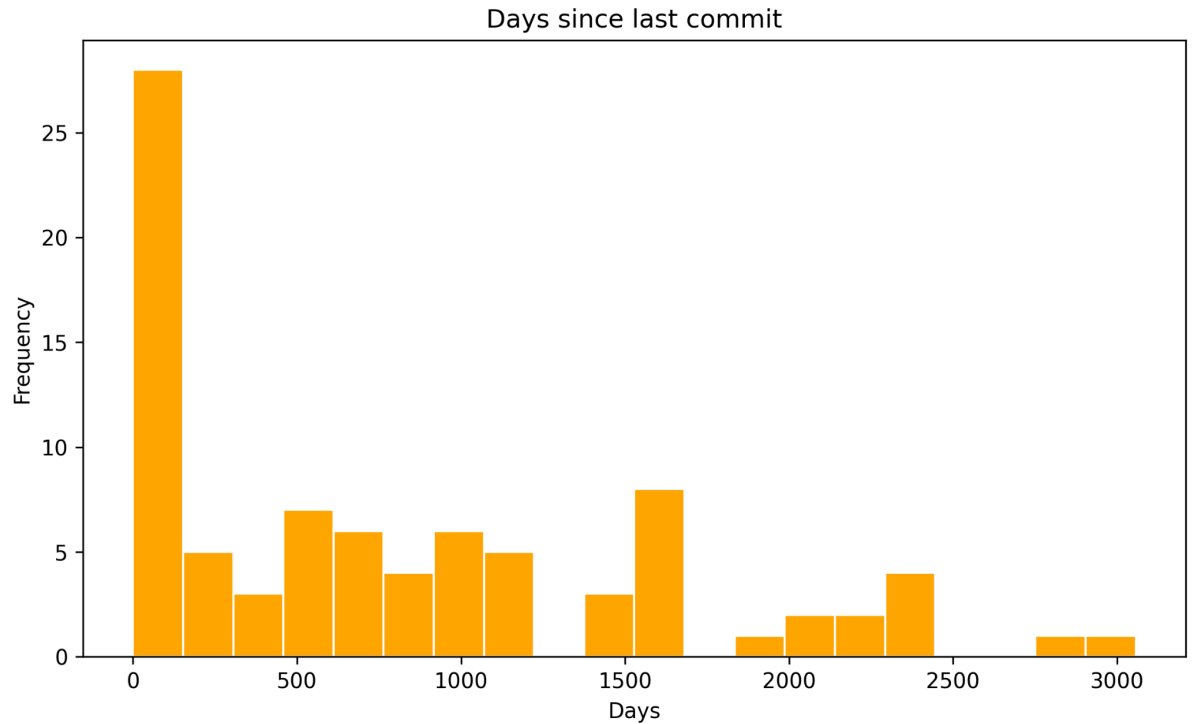


The distribution of stars shows a strong right-skewed pattern. Most repositories have low or intermediate levels of popularity, while only a small group concentrates very high numbers of stars. This indicates that visibility within the dataset is not distributed uniformly.

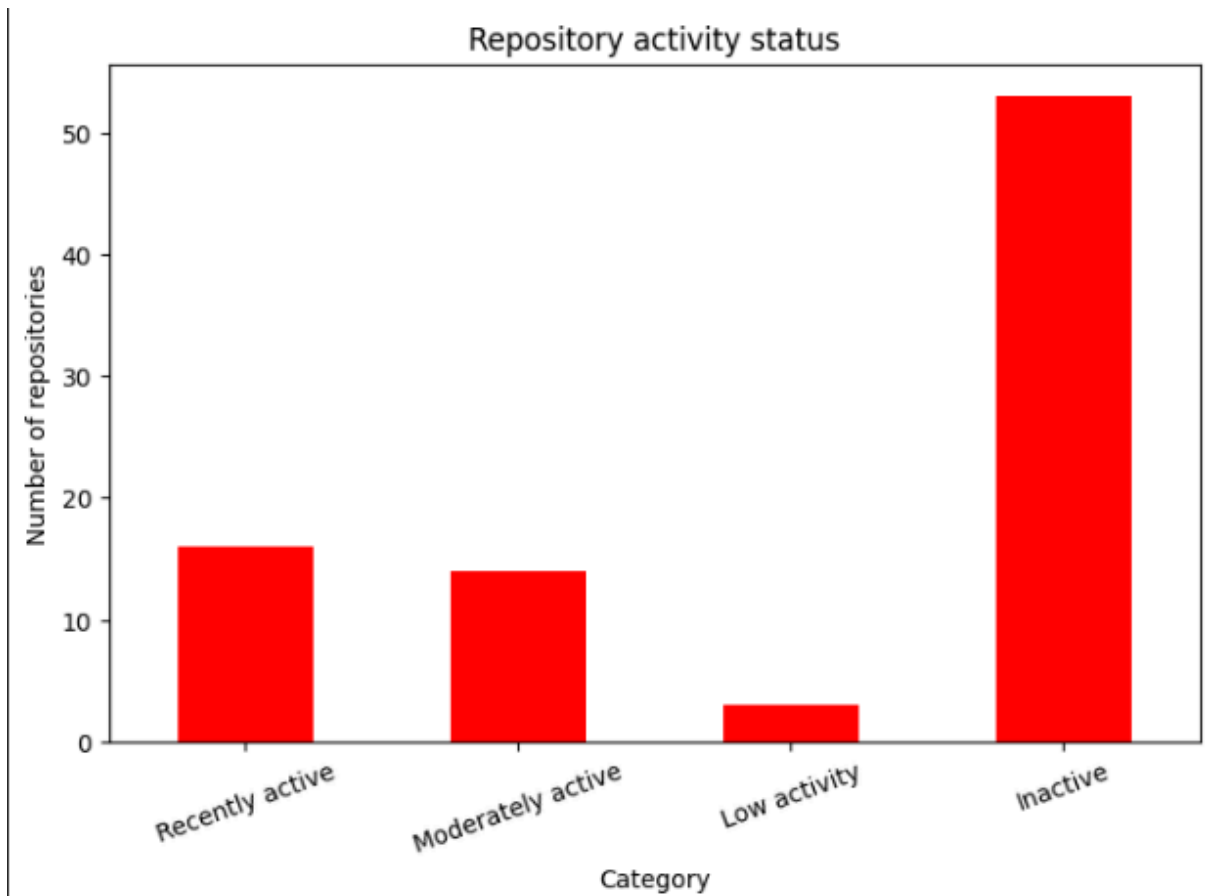


The programming languages chart shows a clear predominance of Python within the dataset, followed by a significant number of repositories with no declared language

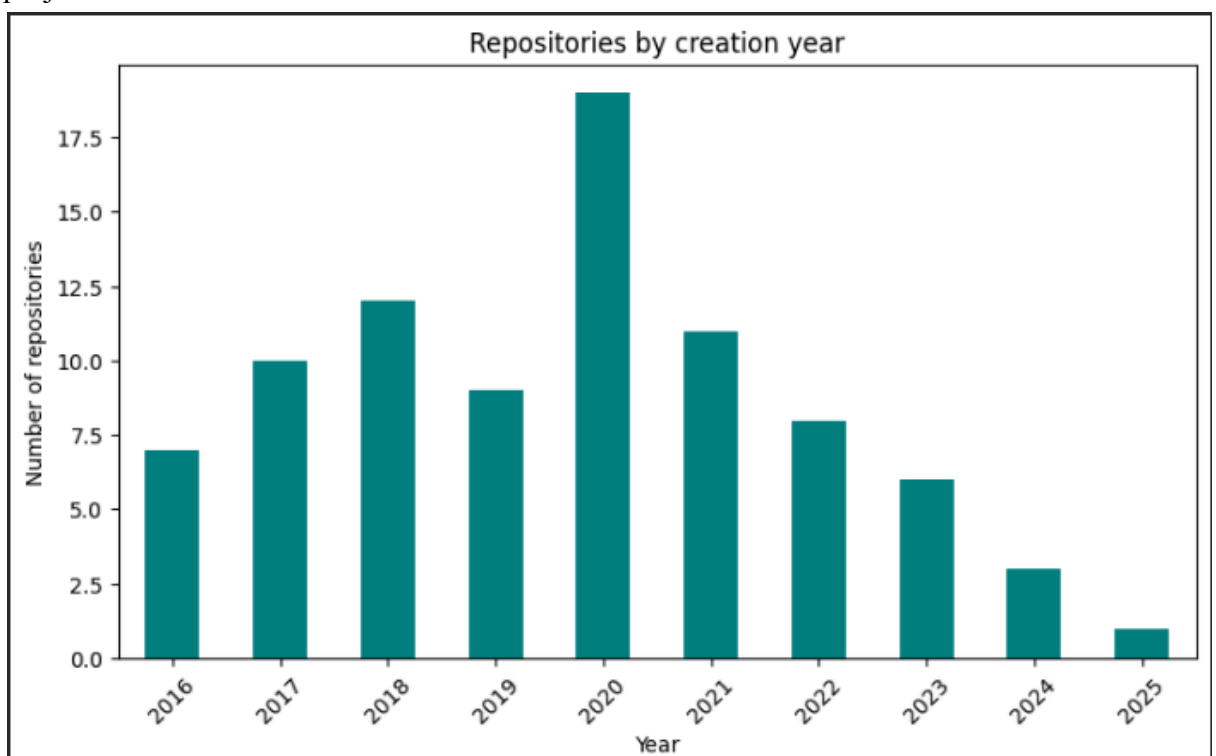
and by Jupyter Notebook. This is consistent with the type of ecosystem analyzed, since Python is the most widely used language in artificial intelligence and scientific software projects.



The distribution of days since the last commit shows that some repositories still exhibit recent activity, but there is also a considerable number of projects with long periods without updates. This suggests that the dataset includes both active tools and repositories with limited or discontinued maintenance.



The activity status classification shows that the predominant category is inactive repositories. To a lesser extent, there are repositories with recent, moderate, and low activity. This result indicates that the collected ecosystem combines recognized projects with different levels of recent maintenance.



The analysis by year of creation shows that the dataset has a higher concentration of repositories created between 2018 and 2021, with 2020 standing out as the year with the highest frequency in the analyzed sample.

8. Feature Matrix Construction:

8.1. Textual Matrix (3034×384):

The affected column was “description.” The textual descriptions of each repository were transformed into dense vectors using the pre-trained model all-MiniLM-L6-v2.

This model was selected because it captures semantic similarity between descriptions, not just word matching. Additionally, it produces 384-dimensional vectors, offering a good balance between representation quality and computational efficiency. This is particularly useful since repository descriptions are usually short but contain important information about the project’s purpose. Moreover, it has a lower computational cost compared to larger models.

8.2. Numerical Variables Matrix (3034×7):

This matrix consists of seven variables that describe the behavior and activity metrics of each repository: stars, forks, days_since_last_commit, repository_age_days, num_topics, ratio_forks_stars, and stars_per_day.

The last two are derived variables:

- ratio_forks_stars measures the level of active usage relative to passive interest.
- stars_per_day captures the growth rate independently of the repository’s age.

All variables were normalized using StandardScaler due to their significantly different ranges. For example, the stars variable can take much larger values than num_topics, which could bias the analysis if scaling is not applied.

8.3. Categorical Language Matrix (3034×41):

This matrix was constructed using one-hot encoding applied to the language variable. As a result, 41 columns were generated, corresponding to the unique programming languages present in the dataset.

8.4. Categorical Topics Matrix (3034×30):

This matrix was generated using MultiLabelBinarizer, considering the 30 most frequent topics in the dataset. This approach is appropriate because a repository can be associated with multiple topics simultaneously.

For example, a repository may belong to categories such as deep-learning, pytorch, and computer-vision. The selection was limited to the top 30 topics to capture the most representative themes of the domain while avoiding noise from less frequent topics.

8.5. Final Combined Matrix (3034×128):

This matrix integrates all previous representations. It includes:

- 50 components obtained from dimensionality reduction applied to textual embeddings
- 7 normalized numerical variables
- 41 categorical language variables
- 30 categorical topic variables

This final representation enables each repository to be described from multiple perspectives: semantic content, quantitative behavior, primary programming language, and thematic areas.

It serves as the main input for subsequent analysis stages, such as clustering or recommendation systems.

9. Dimensionality Reduction:

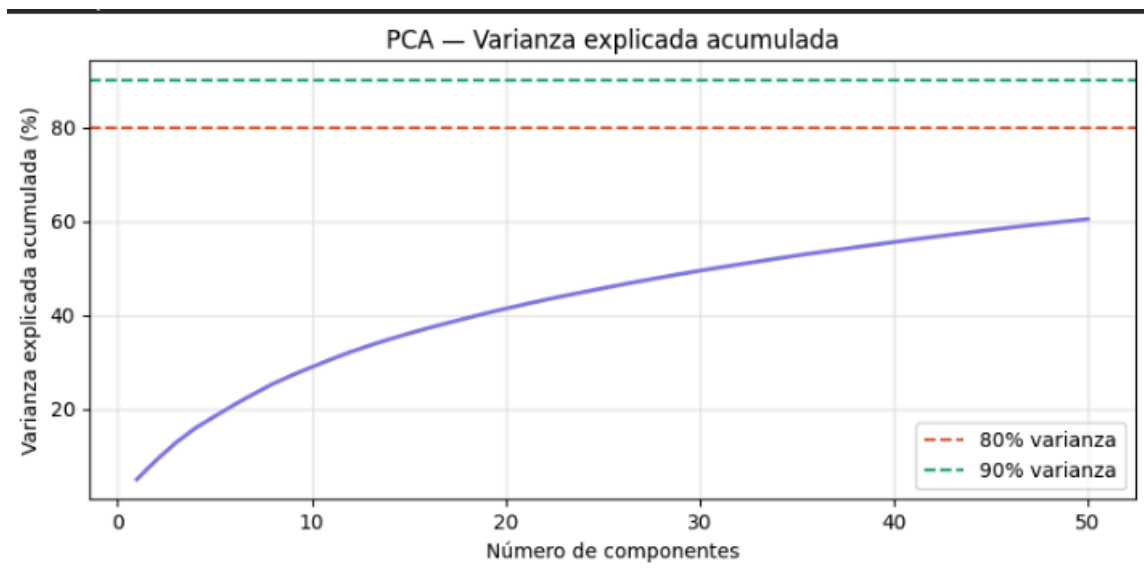
Dimensionality reduction was applied exclusively to the textual embeddings matrix, as it represents the component with the highest number of dimensions in the dataset.

9.1. Application of PCA on embeddings:

PCA was applied to the embeddings matrix of dimension (3034×384) , generating different component configurations to evaluate the amount of retained information.

The results were as follows:

```
Varianza explicada acumulada:  
10 componentes: 29.0%  
20 componentes: 41.4%  
50 componentes: 60.5%
```

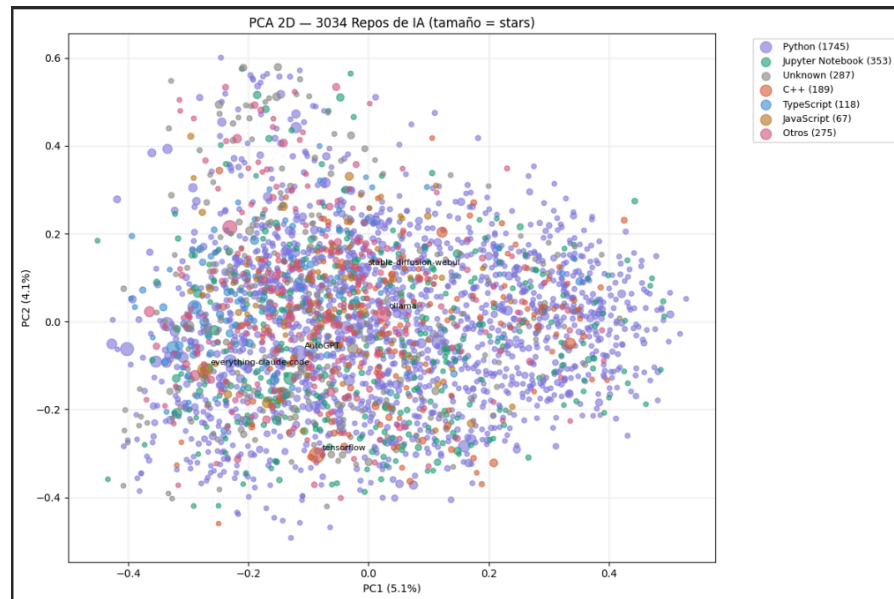


Selection of the Number of Components:

A total of 50 principal components were retained because they reduce the dimensionality of the embeddings from 384 to 50 variables, while preserving approximately 60.74% of the cumulative variance.

Although a high threshold is not reached, this choice is justified as a practical decision, as it improves computational efficiency, reduces dataset complexity, and retains a relevant portion of the semantic information for subsequent stages such as clustering or initial modeling.

9.2. PCA Visualization with 2 Components:



This projection allows us to observe the general distribution of the repositories; however, since it captures only 9.2% of the variance, it does not show a clear separation between groups.

9.3. Integration into the Final Matrix:

The PCA-reduced embeddings (3034×50) were subsequently combined with:

- normalized numerical variables
- categorical language variables
- categorical topic variables

```
Feature matrix completa: (3034, 128)
50 embeddings PCA
7 numéricas normalizadas
41 lenguajes one-hot
30 topics multi-label
Total: 128 features por repo
```


9.4. Comparison Table:

Representation	Dimensions	Explained Variance	Use in the Project
Original embeddings	384	100%	Full textual representation
PCA 2 components	2	9.2%	2D visualization
PCA 10 components	10	29.0%	Preliminary exploration
PCA 20 components	20	41.4%	Information retention comparison
PCA 50 components	50	60.5%	Selected representation for modeling
Final feature matrix	128	Not applicable	Final combined matrix

The table compares the original embedding matrix with different PCA configurations. It can be observed that PCA with 2 components retains only 9.2% of the variance, so it is used solely for visualization purposes. In contrast, PCA with 50 components preserves 60.5% of the explained variance, reducing the dimensionality from 384 to 50 components. Therefore, this configuration was selected to be integrated into the final feature matrix.

Reconstruction Error

```
embeddings_reconstructed = pca_50.inverse_transform(embeddings_pca50)

reconstruction_error = np.mean((embeddings - embeddings_reconstructed) ** 2)

print(f"Error de reconstrucción PCA 50: {reconstruction_error:.6f}")

Error de reconstrucción PCA 50: 0.001144
```

The reconstruction error was calculated for PCA with 50 components, resulting in a value of 0.001144.

This value represents the average difference between the original embeddings and their reconstruction from the principal components. A low error indicates that the dimensionality reduction preserves the original information effectively.

9.5. Technical interpretation of results:

- The original embeddings have high dimensionality, as each description is represented by 384 variables.
- After applying PCA, the dimensionality was reduced from 384 to 50 components, preserving approximately 60.74% of the cumulative variance.
- This indicates that there is some redundancy in the embeddings, since part of the information can be summarized into fewer components.
- The 2D PCA visualization between PC1 and PC2 explains only a small portion of the variance, which is why it does not show clearly separated clusters.
- The final feature matrix improves representation quality by combining text, numerical metrics, language, and topics.
- This final representation is ready for subsequent stages such as clustering or modeling.

Week 7

10. Clustering and Validation Report

In this section, the repositories are **segmented** with the objective of identifying patterns related to **popularity, activity, programming language, and associated topics**. This segmentation helps us better understand the structure of the **artificial intelligence repository ecosystem** and serves as a basis for later stages of the project, such as **ranking, recommendation, or discovery of similar projects**.

For the main clustering process, we used a **mixed representation** composed of **numerical variables, dominant programming language, and topics**. The numerical variables capture the repository's **popularity, activity, age, and growth**. The programming language was encoded using **One-Hot Encoding**, while the topics were encoded using **MultiLabelBinarizer**.

This representation was selected because the objective is not to group repositories only by **textual similarity**, but to segment them according to general patterns of **behavior, technology, and thematic focus**. In this way, each repository is represented as one row in the **feature matrix**, and each column corresponds to a characteristic used by the model.

10.1 Building features for clustering

The feature matrix for clustering combines three types of representations: 7 normalized numerical features that capture popularity and temporal behavior, 41 language columns encoded using One-Hot Encoding, and 30 topic columns encoded with MultiLabelBinarizer based on the 30 most frequent topics.

As a result, the final matrix contains 78 features per repository.

PCA de 10 componentes principales:

```
from sklearn.preprocessing import MultiLabelBinarizer
import ast

# Features numéricas
VARS = ["stars", "forks", "dias_sin_commit",
        "antigüedad_dias", "num_topics",
        "ratio_forks_stars", "stars_por_dia"]

scaler = StandardScaler()
X_num = scaler.fit_transform(df[VARS])

# Encoding de lenguaje
X_lang = pd.get_dummies(df["lenguaje"], prefix="lang").astype(int).values

# Encoding de topics
df["topics_list"] = df["topics"].apply(
    lambda x: ast.literal_eval(x) if isinstance(x, str) else x
)
from collections import Counter
todos_topics = [t for topics in df["topics_list"] for t in topics]
top_topics = [t for t, _ in Counter(todos_topics).most_common(30)]
mlb = MultiLabelBinarizer(classes=top_topics)
X_topics = mlb.fit_transform(df["topics_list"])

# Combinamos todo
X = np.hstack([X_num, X_lang, X_topics])

print(f"Features numéricas: {X_num.shape}")
print(f"Features lenguaje: {X_lang.shape}")
print(f"Features topics: {X_topics.shape}")
print(f"Matrix combinada: {X.shape}")

Features numéricas: (3034, 7)
Features lenguaje: (3034, 41)
Features topics: (3034, 30)
Matrix combinada: (3034, 78)
```

10.2 K Means Analysis

```
1 # =====
2 # K-MEANS - Implementación matemática completa
3 # =====
4
5 def squared_distances(X, centroids):
6     """
7     Calculamos la distancia euclidiana al cuadrado entre cada punto y cada centroide.
8     X shape: (n_samples, n_features)
9     centroids shape: (k, n_features)
10    output shape: (n_samples, k)
11    """
12    diff = X[:, None, :] - centroids[None, :, :]
13    return np.sum(diff**2, axis=2)
14
15 def initialize_centroids(X, k, seed=42):
16     """
17     Inicializa K centroides eligiendo K puntos al azar del dataset.
18     """
19    rng = np.random.default_rng(seed)
20    idx = rng.choice(X.shape[0], size=k, replace=False)
21    return X[idx].copy()
22
23 def assign_clusters(X, centroids):
24     """
25     Asigna cada punto al centroide más cercano.
26     Retorna array de labels con el índice del centroide más cercano.
27     """
28    distances = squared_distances(X, centroids)
29    return np.argmin(distances, axis=1)
30
31 def update_centroids(X, labels, k, previous_centroids):
32     """
33     Actualiza cada centroide moviéndolo al promedio de sus puntos asignados.
34     Si un cluster queda vacío, mantiene el centroide anterior.
35     """
36    new_centroids = previous_centroids.copy()
37    for i in range(k):
38        members = X[labels == i]
39        if len(members) > 0:
40            new_centroids[i] = members.mean(axis=0)
41    return new_centroids
42
43 def compute_inertia(X, labels, centroids):
44     """
45     Calcula la inercia: suma de distancias al cuadrado de cada punto a su centroide.
46     Mide qué tan compactos son los clusters - menor es mejor.
47     """
48    residuals = X - centroids[labels]
49    return np.sum(residuals**2)
50
51 def kmeans_from_scratch(X, k, max_iters=100, tol=1e-4, seed=42):
52     """
53     Algoritmo K-means completo desde cero.
54     Guarda el historial de cada iteración para análisis.
55     """
```

```

51 def kmeans_from_scratch(X, k, max_iters=100, tol=1e-4, seed=42):
52     """
53     Algoritmo K-means completo desde cero.
54     Guarda el historial de cada iteración para análisis.
55     """
56     centroids = initialize_centroids(X, k, seed=seed)
57     initial_centroids = centroids.copy()
58     history = []
59
60     for iteration in range(max_iters):
61         labels = assign_clusters(X, centroids)
62         inertia_before = compute_inertia(X, labels, centroids)
63         new_centroids = update_centroids(X, labels, k, centroids)
64         labels_after = assign_clusters(X, new_centroids)
65         centroid_shift = float(np.linalg.norm(new_centroids - centroids))
66
67         history.append({
68             "iteration": iteration,
69             "inertia": inertia_before,
70             "centroid_shift": centroid_shift,
71             "centroids_before": centroids.copy(),
72             "centroids_after": new_centroids.copy(),
73             "labels_before": labels.copy(),
74             "labels_after": labels_after.copy()
75         })
76
77         centroids = new_centroids
78         if centroid_shift < tol or np.array_equal(labels, labels_after):
79             print(f"Convergió en iteración {iteration + 1}")
80             break
81
82     labels = assign_clusters(X, centroids)
83     inertia = compute_inertia(X, labels, centroids)
84
85     return {
86         "labels": labels,
87         "centroids": centroids,
88         "initial_centroids": initial_centroids,
89         "inertia": inertia,
90         "history": history
91     }
92
93 print("Funciones K-means definidas correctamente.")
94 print("Funciones disponibles:")
95 print(" - squared_distances(X, centroids)")
96 print(" - initialize_centroids(X, k, seed)")
97 print(" - assign_clusters(X, centroids)")
98 print(" - update_centroids(X, labels, k, previous_centroids)")
99 print(" - compute_inertia(X, labels, centroids)")
100 print(" - kmeans_from_scratch(X, k, max_iters, tol, seed)")

```

```

Funciones K-means definidas correctamente.
Funciones disponibles:
- squared_distances(X, centroids)
- initialize_centroids(X, k, seed)
- assign_clusters(X, centroids)
- update_centroids(X, labels, k, previous_centroids)
- compute_inertia(X, labels, centroids)
- kmeans_from_scratch(X, k, max_iters, tol, seed)

```

K-means was implemented from scratch using only NumPy. The algorithm is composed of four main mathematical functions:

- **squared_distances** calculates the squared Euclidean distance between each point and each centroid using broadcasting. This avoids explicit loops and makes the computation more efficient.
- **assign_clusters** assigns each point to the nearest centroid by applying `argmin` over the distance matrix.
- **update_centroids** recalculates each centroid as the arithmetic mean of all the points assigned to that cluster.
- **compute_inertia** calculates the sum of squared distances from each point to its assigned centroid. This metric measures the compactness of the clusters.

The algorithm iterates until convergence, which is defined as a centroid movement smaller than $1e-4$ or no changes in the cluster assignments.

10.3 Reducción PCA antes del clustering

After building the feature matrix `X`, we applied PCA to reduce the dimensionality from 78 original variables to 10 principal components.

This reduction was performed before clustering because K-means works with Euclidean distances and can be affected by high-dimensional spaces, especially when combining numerical variables, encoded programming languages, and binary topic features.

```

1 # Reducir dimensiones con PCA antes de clustering
2 # El profesor usó n_components=10 - hacemos lo mismo
3 pca = PCA(n_components=10, random_state=42)
4 X_embed = pca.fit_transform(X)
5
6 varianza = np.cumsum(pca.explained_variance_ratio_) * 100
7 print("Varianza explicada acumulada con PCA:")
8 for i, v in enumerate(varianza):
9     print(f"  PC{i+1}: {v:.1f}%")
10
11 print(f"\nShape después de PCA: {X_embed.shape}")
12 print(f"Reducimos de {X.shape[1]} features a {X_embed.shape[1]}")

```

```

... Varianza explicada acumulada con PCA:
  PC1: 21.2%
  PC2: 38.9%
  PC3: 49.6%
  PC4: 58.2%
  PC5: 65.5%
  PC6: 70.9%
  PC7: 73.9%
  PC8: 76.5%
  PC9: 78.2%
  PC10: 79.7%

Shape después de PCA: (3034, 10)
Reducimos de 78 features a 10

```

We used 10 principal components because they allow us to reduce the matrix from 78 dimensions to 10 dimensions, while preserving approximately 79.7% of the cumulative variance.

Additionally, PC1 and PC2 explain 38.9% of the variance together, so they can be used as a visual support to observe the general distribution of the repositories.

However, this visualization should not be interpreted as definitive evidence of cluster separation, because PCA preserves variance, but it does not directly optimize the separation between groups.

10.4 Probar Kmeans con $K=2$ y compararlo con sklearn

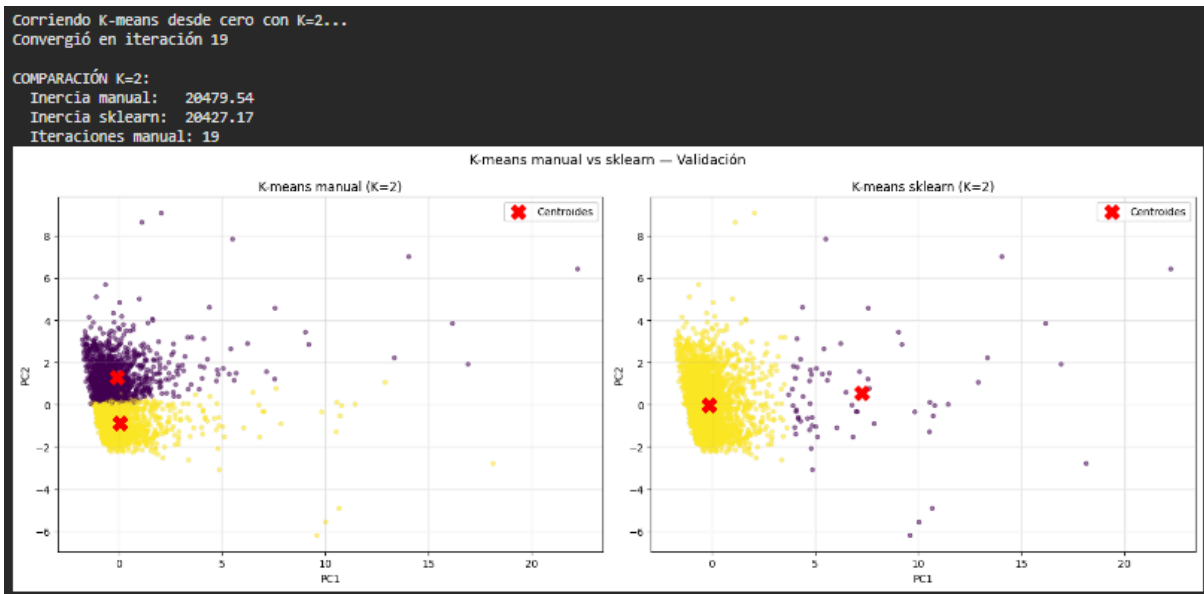
We validate the manual implementation by running K-means with $K = 2$ and comparing the resulting inertia with sklearn.

If both inertia values are similar, it means that the mathematical implementation is correct. Small differences are normal because sklearn uses K-means++ initialization, which is more sophisticated than simple random initialization.

```

1 from sklearn.cluster import KMeans
2 from sklearn.metrics import silhouette_score
3 import os
4
5 os.makedirs("reports", exist_ok=True)
6
7 # Correr K-means desde cero con K=2
8 print("Corriendo K-means desde cero con K=2...")
9 results = kmeans_from_scratch(X_embed, k=2)
10
11 # Correr K-means sklearn con K=2
12 kmeans_sk = KMeans(n_clusters=2, random_state=42, n_init=100)
13 kmeans_sk.fit(X_embed)
14
15 print(f"\nCOMPARACIÓN K=2:")
16 print(f" Inercia manual: {results['inertia']:.2f}")
17 print(f" Inercia sklearn: {kmeans_sk.inertia_.2f}")
18 print(f" Iteraciones manual: {len(results['history'])}")
19
20 # Visualizar
21 fig, axes = plt.subplots(1, 2, figsize=(16, 6))
22
23 # Manual
24 axes[0].scatter(X_embed[:, 0], X_embed[:, 1],
25                c=results["labels"], cmap="viridis", alpha=0.5, s=15)
26 axes[0].scatter(results["centroids"][:, 0], results["centroids"][:, 1],
27                color="red", marker="x", s=200, label="Centroides")
28 axes[0].set_title("K-means manual (K=2)")
29 axes[0].set_xlabel("PC1")
30 axes[0].set_ylabel("PC2")
31 axes[0].legend()
32 axes[0].grid(alpha=0.3)
33
34 # Sklearn
35 axes[1].scatter(X_embed[:, 0], X_embed[:, 1],
36                c=kmeans_sk.labels_, cmap="viridis", alpha=0.5, s=15)
37 axes[1].scatter(kmeans_sk.cluster_centers_[:, 0], kmeans_sk.cluster_centers_[:, 1],
38                color="red", marker="x", s=200, label="Centroides")
39 axes[1].set_title("K-means sklearn (K=2)")
40 axes[1].set_xlabel("PC1")
41 axes[1].set_ylabel("PC2")
42 axes[1].legend()
43 axes[1].grid(alpha=0.3)
44
45 plt.suptitle("K-means manual vs sklearn - Validación", fontsize=13)
46 plt.tight_layout()
47 plt.savefig("reports/kmeans_validacion.png", dpi=150, bbox_inches="tight")
48 plt.show()
49 print("Guardado en reports/kmeans_validacion.png")

```

What does converging in 19 iterations mean?

Imagine we have 2 centroids and the algorithm moves them step by step:

- In iteration 1, the centroids move a lot because they are still far from their optimal position.
- In iteration 2, they still move considerably because they are adjusting.
- By iteration 5, they move less and start becoming more stable.
- By iteration 10, they move very little.
- By iteration 19, the centroids no longer move significantly, meaning they have reached their final position.

When the centroids stop moving, the algorithm concludes: “I have already found the groups, so there is no need to continue.” This process is called convergence.

- In our case, the manual implementation converged in 19 iterations with an inertia of 20,479, which is almost identical to sklearn’s inertia of 20,427.
- The small difference is explained by the initialization method: sklearn uses K-means++, which selects initial centroids that are more separated from each other, slightly reducing the final inertia.
- This confirms that our mathematical implementation from scratch is correct..

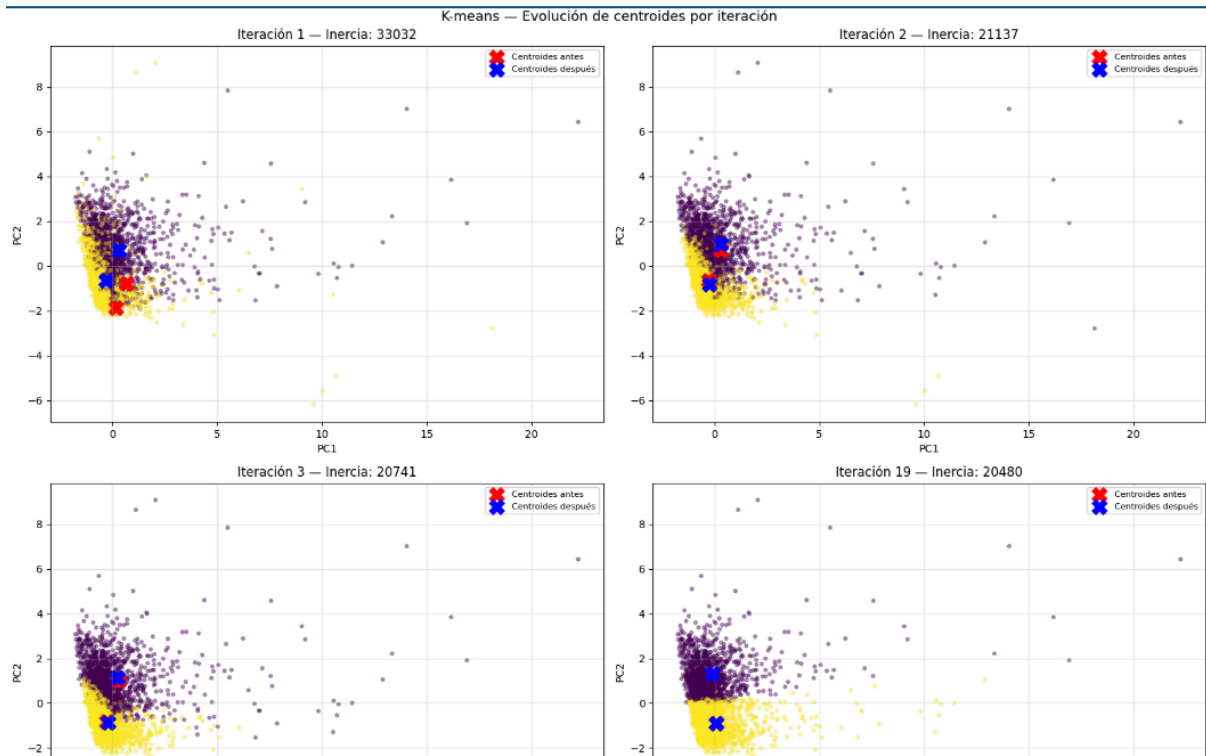
10.5 Visualizar iteraciones del algoritmo

The first iterations of the algorithm were visualized to show convergence. In each iteration, the red centroids represent the position before the update, while the blue centroids represent the position after the update.

The inertia decreases in each iteration as the centroids move closer to the real average of their assigned points. In the final iteration, the red and blue centroids overlap, which indicates convergence.

When this difference is smaller than 0.0001, the algorithm converges and stops.

```
3 # Visualizar primeras 4 iteraciones para mostrar cómo converge
4 fig, axes = plt.subplots(2, 2, figsize=(16, 12))
5 axes = axes.flatten()
6
7 iteraciones_a_mostrar = [0, 1, 2, len(results["history"])-1]
8
9 for idx, iter_num in enumerate(iteraciones_a_mostrar):
10     ax = axes[idx]
11     iter_data = results["history"][iter_num]
12
13     ax.scatter(X_embed[:, 0], X_embed[:, 1],
14               c=iter_data["labels_before"],
15               cmap="viridis", alpha=0.4, s=10)
16     ax.scatter(iter_data["centroids_before"][:, 0],
17               iter_data["centroids_before"][:, 1],
18               color="red", marker="X", s=200,
19               label="Centroides antes")
20     ax.scatter(iter_data["centroids_after"][:, 0],
21               iter_data["centroids_after"][:, 1],
22               color="blue", marker="X", s=200,
23               label="Centroides después")
24     ax.set_title(f"Iteración {iter_num + 1} - Inercia: {iter_data['inertia']:.0f}")
25     ax.set_xlabel("PC1")
26     ax.set_ylabel("PC2")
27     ax.legend(fontsize=8)
28     ax.grid(alpha=0.3)
29
30 plt.suptitle("K-means - Evolución de centroides por iteración", fontsize=13)
31 plt.tight_layout()
32 plt.savefig("reports/kmeans_iteraciones.png", dpi=150, bbox_inches="tight")
33 plt.show()
34 print("Guardado en reports/kmeans_iteraciones.png")
```



The visualizations confirm the progressive convergence of the algorithm. In iteration 1, the red and blue centroids are separated, meaning the centroid moves significantly toward the real average of its assigned points.

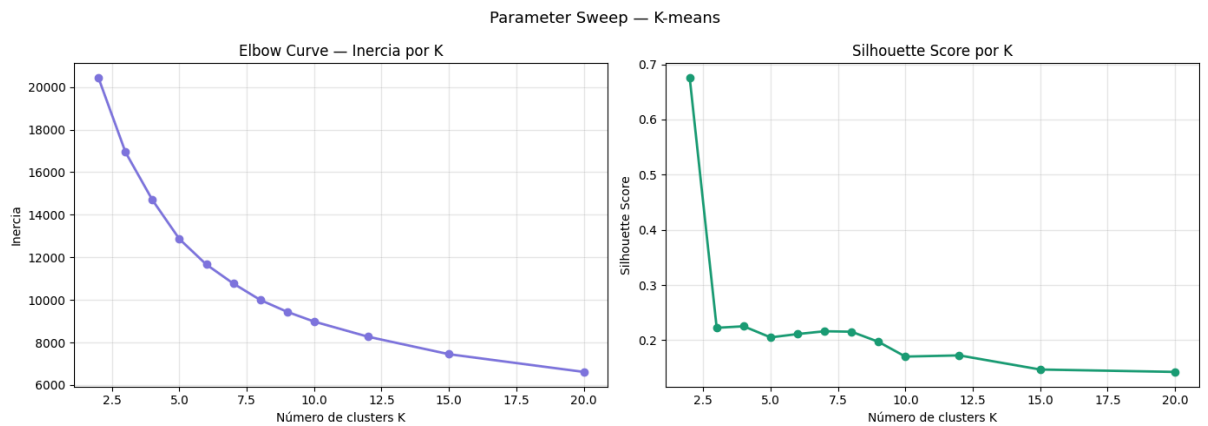
In the intermediate iterations, the movement becomes smaller. In the final iteration, both centroids overlap, confirming that the algorithm found a stable solution.

10.6 Parameter sweep K-means

Different values of K between 2 and 20 were evaluated to analyze how the quality of clustering changes depending on the number of groups.

For each value of K, two main metrics were calculated:

- **Inertia:** measures the sum of internal distances between the points and their centroids. The lower the inertia, the more compact the clusters are.
- **Silhouette Score:** measures how well each point is separated from other clusters. Higher values indicate better separation between groups.



K	Inercia	Silhouette	Iteraciones
2	20427.38	0.675	11
3	16941.86	0.222	32
4	14692.44	0.225	10
5	12860.63	0.205	15
6	11663.22	0.211	27
7	10770.83	0.216	36
8	9999.19	0.215	15
9	9439.36	0.197	79
10	8981.64	0.170	20
12	8270.38	0.172	20
15	7449.59	0.147	28
20	6617.28	0.142	27

- The parameter sweep shows that $K = 2$ produces the highest Silhouette Score, with a value of 0.675. However, this result is too broad for thematic analysis because it divides the ecosystem into only two large groups.
- From $K = 3$ onward, the Silhouette Score stabilizes around 0.22, indicating diffuse boundaries between clusters. This is expected in real-world text-based data, where thematic categories naturally overlap.
- Between $K = 3$ and $K = 5$, the reduction in inertia begins to flatten, meaning that adding more clusters provides less additional improvement.
- Therefore, $K = 5$ was selected as the optimal balance between thematic interpretability and mathematical justification.

10.7 K-means final con $K=5$

```

1 # K-means final con K=5
2 print("Corriendo K-means final con K=5...")
3 results_k5 = kmeans_from_scratch(X_embed, k=5)
4
5 # Validar con sklearn
6 kmeans_final = KMeans(n_clusters=5, random_state=42, n_init=100)
7 kmeans_final.fit(X_embed)
8
9 print(f"Convergió en iteración: {len(results_k5['history'])}")
10 print(f"Inercia manual: {results_k5['inertia']:.2f}")
11 print(f"Inercia sklearn: {kmeans_final.inertia_:.2f}")
12 print(f"Silhouette: {silhouette_score(X_embed, kmeans_final.labels_):.3f}")
13
14 # Guardar labels en dataframe
15 df["cluster_kmeans"] = kmeans_final.labels_
16
17 print(f"\nDistribución de repos por cluster:")
18 print(df["cluster_kmeans"].value_counts().sort_index())

```

```

• Corriendo K-means final con K=5...
  Convergió en iteración 33
  Convergió en iteración: 33
  Inercia manual: 13888.24
  Inercia sklearn: 12845.53
  Silhouette: 0.205

  Distribución de repos por cluster:
  cluster_kmeans
0      1255
1       930
2       788
3        56
4         5

```

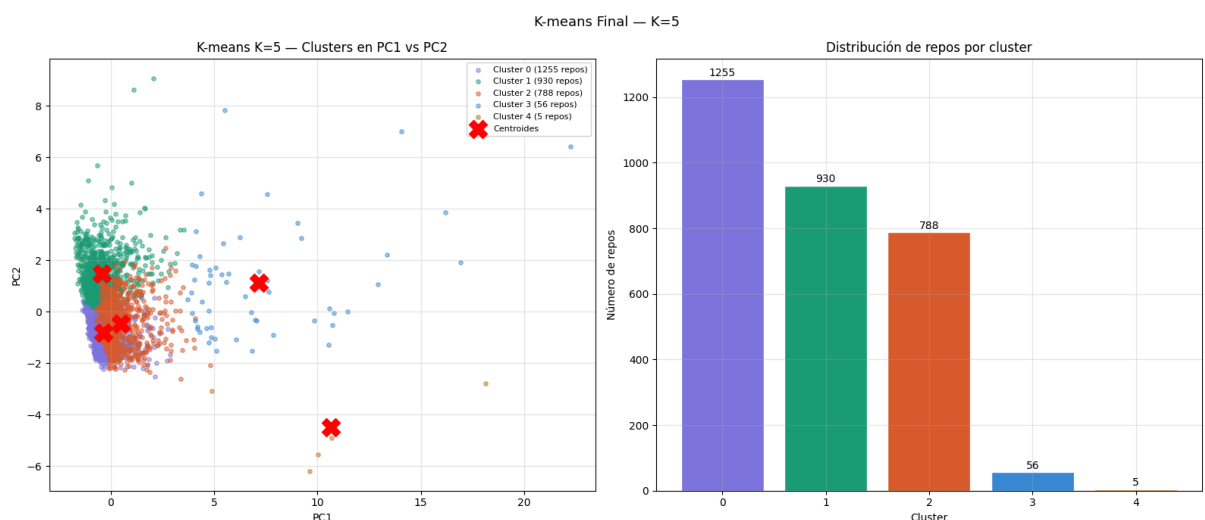
- **K-means with K = 5** reveals an **asymmetric distribution**.
- Clusters **0, 1, and 2** contain most of the repositories, while **cluster 3** has **56 repositories** and **cluster 4** has **only 5 repositories**.
- This asymmetry suggests that the ecosystem has a **dominant central structure**, with **peripheral groups of highly specialized repositories**.
- The difference in inertia between the **manual implementation (13,888)** and **sklearn (12,845)** is larger than with **K = 2**, confirming that when **K is higher**, the **centroid initialization** has a stronger impact on the final result.

10.8 Visualizar clusters finales K=5

```

1 fig, axes = plt.subplots(1, 2, figsize=(16, 7))
2
3 colores = ['#7F77DD', '#1D9E75', '#D85A30', '#3B8BD4', '#BA7517']
4
5 # Gráfico 1 – Clusters en espacio PCA
6 for i in range(5):
7     mask = kmeans_final.labels_ == i
8     axes[0].scatter(X_embed[mask, 0], X_embed[mask, 1],
9                    color=colores[i], alpha=0.5, s=15,
10                   label=f"Cluster {i} ({mask.sum()} repos)")
11
12 axes[0].scatter(kmeans_final.cluster_centers[:, 0],
13                kmeans_final.cluster_centers[:, 1],
14                color='red', marker='X', s=300,
15                zorder=5, label='Centroides')
16 axes[0].set_title("K-means K=5 – Clusters en PC1 vs PC2")
17 axes[0].set_xlabel("PC1")
18 axes[0].set_ylabel("PC2")
19 axes[0].legend(fontsize=8)
20 axes[0].grid(alpha=0.3)
21
22 # Gráfico 2 – Distribución de repos por cluster
23 cluster_counts = df["cluster_kmeans"].value_counts().sort_index()
24 axes[1].bar(cluster_counts.index, cluster_counts.values,
25            color=colores, edgecolor='white')
26 axes[1].set_title("Distribución de repos por cluster")
27 axes[1].set_xlabel("Cluster")
28 axes[1].set_ylabel("Número de repos")
29 for i, v in enumerate(cluster_counts.values):
30     axes[1].text(i, v + 10, str(v), ha='center', fontsize=10)
31 axes[1].grid(alpha=0.3)
32
33 plt.suptitle("K-means Final – K=5", fontsize=13)
34 plt.tight_layout()
35 plt.savefig("reports/kmeans_final.png", dpi=150, bbox_inches="tight")
36 plt.show()
37 print("Guardado en reports/kmeans_final.png")

```



The visualization shows the five clusters obtained with K-means, projected onto the first two principal components, PC1 and PC2.

In the chart on the left, clusters 0, 1, and 2 contain most of the repositories and show some overlap in the central area. This means that many repositories share similar characteristics, so the groups should be interpreted as exploratory segments, not as rigid separations.

Clusters 3 and 4 are much smaller and appear farther from the main group. This suggests that they represent repositories with atypical or extreme characteristics. This idea is confirmed in the distribution chart, where cluster 3 contains 56 repositories and cluster 4 contains only 5 repositories.

As a result, the next step is to analyze the profile of each cluster to identify which variables explain these differences.

10.9 Perfil de cada cluster

The profile analysis shows that the clusters are mainly differentiated by popularity, repository age, recent activity, and growth speed.

Programming language does not explain the separation by itself, since Python appears as the dominant language in almost all groups.

```
=====
PERFIL DE CLUSTERS — K-means K=5
=====

CLUSTER 0 — 1255 repos
Stars promedio:          4,070
Forks promedio:          455
Días sin commit prom:    336
Antigüedad promedio:    1,170 días
Stars por día prom:      6.31
Top 3 lenguajes:
- Python: 806
- Jupyter Notebook: 119
- Unknown: 114
Top 5 repos:
- upstash/context7 (54,137 ★)
- ggml-org/whisper.cpp (49,182 ★)
- janhq/jan (42,272 ★)
- datawhalechina/hello-agents (41,834 ★)
- TencentARC/GFPGAN (37,441 ★)
```

Cluster 0, called “**Modern AI Tools**”, groups 1,255 repositories.

It has an average of 4,070 stars, an average age of 1,170 days, and 336 days since the last commit.

This cluster represents recent, functional tools with intermediate adoption, such as whisper.cpp, GFPGAN, jan, and context7.

```
CLUSTER 1 – 930 repos
Stars promedio:      4,326
Forks promedio:      1,060
Días sin commit prom: 1,055
Antigüedad promedio: 2,933 días
Stars por día prom:   1.39
Top 3 lenguajes:
- Python: 504
- Jupyter Notebook: 144
- C++: 91
Top 5 repos:
- JuliaLang/julia (48,642 ★)
- google-research/bert (40,003 ★)
- floodsung/Deep-Learning-Papers-Reading-Roadmap (39,497 ★)
- naptha/tesseract.js (38,041 ★)
- google-research/google-research (37,825 ★)
```

Cluster 1, called “Established and Educational Projects”, contains 930 repositories.

It has an average of 4,326 stars, but it stands out as the oldest group, with an average age of 2,933 days and 1,055 days since the last commit.

This indicates that the cluster includes consolidated projects, historical references, or educational resources within the ecosystem, such as BERT, paper roadmaps, Julia, and google-research.

```
CLUSTER 2 – 788 repos
Stars promedio:      8,409
Forks promedio:      1,143
Días sin commit prom: 228
Antigüedad promedio: 1,798 días
Stars por día prom:   8.92
Top 3 lenguajes:
- Python: 400
- Unknown: 86
- Jupyter Notebook: 84
Top 5 repos:
- unslothai/unsloth (63,290 ★)
- pathwaycom/llm-app (59,879 ★)
- Mintplex-Labs/anything-llm (59,264 ★)
- mem0ai/mem0 (54,430 ★)
- run-llama/llama_index (49,048 ★)
```

Cluster 2, called “Emerging LLM Ecosystem”, groups 788 repositories.

It has an average of 8,409 stars, 228 days since the last commit, and 8.92 stars per day.

This is a more active and faster-growing group than clusters 0 and 1. It includes projects such as unsloth, llama_index, anything-llm, mem0, and llm-app, so it represents the core of modern projects associated with the LLM boom.

```
CLUSTER 3 – 56 repos
Stars promedio:      84,146
Forks promedio:      18,257
Días sin commit prom: 83
Antigüedad promedio: 2,113 días
Stars por día prom:  78.04
Top 3 lenguajes:
- Python: 32
- Jupyter Notebook: 6
- C++: 5
Top 5 repos:
- tensorflow/tensorflow (194,936 ★)
- Significant-Gravitas/AutoGPT (183,889 ★)
- ollama/ollama (170,350 ★)
- AUTOMATIC1111/stable-diffusion-webui (162,659 ★)
- f/prompts.chat (161,175 ★)
```

Cluster 3, labeled “**Massive and Influential Projects**” contains 56 repositories. It averages 84,146 stars and 18,257 forks, with only 83 days without commits and 78.04 stars gained per day. It represents the ecosystem’s largest, most widely adopted, and influential projects, such as TensorFlow, Auto-GPT, Ollama, and Stable Diffusion WebUI.

```
CLUSTER 4 – 5 repos
Stars promedio:      57,471
Forks promedio:      7,762
Días sin commit prom: 3
Antigüedad promedio: 33 días
Stars por día prom:  1,829.22
Top 3 lenguajes:
- Python: 3
- JavaScript: 1
- TypeScript: 1
Top 5 repos:
- affaan-m/everything-claude-code (170,357 ★)
- JuliusBrussee/caveman (50,578 ★)
- MemPalace/mempalace (50,401 ★)
- kyegomez/OpenMythos (11,179 ★)
- nexu-io/open-design (4,841 ★)
```

Cluster 4, labeled “**Recently Viral Repositories**” contains only 5 repositories, so it should be interpreted with caution. It has an average age of just 33 days, yet it averages 57,471 stars and 1,829.22 stars gained per day. This pattern points to extremely recent repositories that experienced explosive viral growth, such as everything-claude-code, caveman, and mempalace.

In conclusion, the clusters do not solely reflect differences in programming language, but also distinct levels of maturity, impact, and growth. Clusters 0, 1, and 2 represent the ecosystem's main segments, while clusters 3 and 4 capture either highly influential projects or unusual cases of accelerated growth.

DBSCAN:

We will evaluate DBSCAN to compare its clustering performance with K-means. To do so, a parameter sweep was conducted using different values of `eps` and `min_samples`, analyzing the number of clusters generated, the percentage of noise, and the Silhouette Score.

eps	min_samples	clusters	ruido %	silhouette
0.50	5	9	79.40%	0.030
0.50	10	4	87.11%	0.104
0.50	20	1	92.32%	nan
0.50	30	2	93.54%	0.398
0.75	5	18	45.09%	-0.139
0.75	10	4	60.51%	0.054
0.75	20	3	72.05%	0.191
0.75	30	2	76.60%	0.198
1.00	5	8	18.66%	0.031
1.00	10	3	26.50%	0.138
1.00	20	2	38.33%	0.087
1.00	30	1	45.98%	nan
1.25	5	2	8.50%	0.258
1.25	10	1	11.40%	nan
1.25	20	1	16.41%	nan
1.25	30	1	20.70%	nan
1.50	5	1	4.38%	nan
1.50	10	1	5.57%	nan
1.50	20	1	7.65%	nan
1.50	30	1	8.93%	nan
1.75	5	1	2.54%	nan
1.75	10	1	3.23%	nan
1.75	20	1	4.05%	nan
1.75	30	1	4.75%	nan
2.00	5	1	1.98%	nan
2.00	10	1	2.44%	nan
2.00	20	1	2.80%	nan
2.00	30	1	3.07%	nan
2.50	5	2	0.99%	0.734
2.50	10	1	1.29%	nan
2.50	20	1	1.71%	nan
2.50	30	1	1.88%	nan
3.00	5	2	0.69%	0.730
3.00	10	1	0.96%	nan
3.00	20	1	0.99%	nan
3.00	30	1	1.09%	nan
3.50	5	1	0.53%	nan

eps=1.25 and min_samples=5 were selected because this configuration provided the most balanced results within the parameter sweep. With lower eps values, the model generated more clusters but classified too many repositories as noise; for instance, with eps=0.50, the noise level exceeded 79%. In contrast, with higher eps values, DBSCAN merged nearly all repositories into a single cluster, reducing its usefulness for meaningful segmentation.

```

Etiquetas encontradas: [np.int64(-1), np.int64(0), np.int64(1)]
Número de clusters sin contar ruido: 2
Porcentaje de ruido: 8.50%
Etiquetas sin ruido: [np.int64(0), np.int64(1)]
Silhouette sin ruido: 0.258

Distribución de etiquetas DBSCAN:
-1      258
 0     2771
 1         5

```

With the selected configuration, DBSCAN identified 2 actual clusters and labeled 258 repositories as noise, representing 8.50% of the dataset. Cluster 0 contained the vast majority of repositories, with 2,771 cases, while Cluster 1 grouped only 5 repositories.

The Silhouette Score, excluding noise points, was 0.258. Although this value is slightly higher than the one obtained with K-means, DBSCAN's distribution is highly unbalanced. This suggests that the algorithm failed to form several comparable groups; instead, it clustered almost the entire dataset into a single group while isolating a very small set of atypical repositories.

Comparación entre K-means y DBSCAN

Model	Parameters	Clusters	Silhouette Score	Inertia	Density / Noise Metric	Distribution
K-means	K=5	5	0.205	12845.53	No aplica	Cluster 0 = 1255; Cluster 1 = 930; Cluster 2 = 788; Cluster 3 = 56; Cluster 4 = 5
DBSCAN	eps=1.25, min_samples=5	2	0.258	No aplica	8.50% de ruido	Ruido = 258; Cluster 0 = 2771; Cluster 1 = 5

The table shows that K-means produces five interpretable clusters, although with a moderately low Silhouette Score of 0.205. DBSCAN achieves a slightly higher Silhouette Score of 0.258, but its distribution is highly unbalanced, as it places 2,771 repositories into a single cluster while separating only 5 repositories into another.

For this reason, K-means remains the primary algorithm for constructing interpretable repository profiles. DBSCAN was also evaluated as a secondary clustering approach, in this case density-based, because it allows the analysis of naturally dense groups and points identified as noise. However, its results were not selected as the final

segmentation because it produced a highly unbalanced distribution. This comparison highlights that a slightly higher metric, such as DBSCAN's Silhouette Score, does not necessarily imply a more useful segmentation for the project's objective.

Failure Analysis

- For K-means, the Silhouette Score was 0.205, indicating a moderate-to-low level of separation between clusters. In addition, the distribution was unbalanced: Clusters 0, 1, and 2 contain the majority of repositories, while Clusters 3 and 4 capture only a small number of atypical cases.
- It is also evident that Python dominates across nearly all clusters, suggesting that programming language alone does not explain the segmentation. Instead, the main differences are driven by popularity, repository age, recent activity, and growth rate.
- For DBSCAN, although the Silhouette Score was higher at 0.258, the segmentation was poorly balanced: 2,771 repositories were assigned to a single cluster, 5 to another, and 258 were labeled as noise. This indicates that DBSCAN was unable to identify several comparable dense groups within this representation of the data.

Conclusions on K-means and DBSCAN

For K-means, five clusters were selected because they provide a balance between inertia reduction, an acceptable Silhouette Score, and meaningful group interpretability. Based on these clusters, the following repository profiles were identified: **modern AI tools, established and educational projects, the emerging LLM ecosystem, massive and influential projects, and recently viral repositories.**

DBSCAN was also evaluated as a secondary clustering algorithm. However, it was not selected as the final segmentation because it grouped almost all repositories into a single cluster while isolating only a very small group. Although it achieved a slightly higher **Silhouette Score**, its distribution was less useful for profile-based analysis.

In conclusion, **K-means proved to be more suitable for the project's objective**, while DBSCAN served as a valuable comparison to examine the dataset's behavior from a density-based perspective.

Week 10

4 Recommendation, Ranking, or Predictive Decision Engine

4.1 Baseline system

The baseline recommends the most popular repositories by star count, regardless of the query repository. This is the simplest possible system and serves as the mandatory reference point without a baseline there is no way to determine whether a more complex system adds real value.

The baseline has an obvious structural limitation: the most starred repositories such as tensorflow, pytorch, and transformers belong to different technical areas. A developer working on a speech recognition project would receive recommendations dominated by general-purpose frameworks unrelated to their domain. Popularity does not imply topical similarity.

Baseline Precision@10: 0.108, meaning that on average only 1 out of every 10 recommended repositories shares at least 2 topics with the query repository.

System	Precision@10	Interpretation
Baseline (popularity)	0.108	Just recommend the most famous repos.
Content-based TF-IDF	0.274	Convert every repo in a vector of numbers
SVD matrix factorization	0.680	Every row is a repo and every column is a topic.

- El baseline alcanza solo 0.108, 1 de cada 10 recomendaciones es relevante.
- El sistema TF-IDF duplica ese resultado llegando a 0.274.
- El sistema SVD con matrix factorization alcanza 0.680, casi 7 de cada 10 recomendaciones son temáticamente relevantes.

La mejora de SVD sobre el baseline con una diferencia de 57%.

4.2 Stronger system

The stronger system combines three similarity signals into a hybrid ranking score.

- TF-IDF:content similarity: each repository description is represented as a weighted term vector using TF-IDF with a vocabulary of 5,000 terms. Cosine similarity between vectors measures textual proximity. Two repositories with similar vocabulary in their descriptions receive a high content score.
- PCA structural similarity: repositories are projected into a 10-component PCA space built from the combined feature matrix of numeric variables, language encoding, and topic encoding. Cosine similarity in this space captures structural proximity beyond text.

- K-means cluster bonus: repositories belonging to the same cluster receive a bonus signal because the clustering validated that they share thematic and behavioral characteristics.

How do the three signals combine?

$$\text{score_final} = 0.5 \times \text{TF-IDF} + 0.3 \times \text{PCA} + 0.2 \times \text{cluster_bonus}$$

The final hybrid score is a weighted combination of these three signals, prioritizing textual content over structural features. Content-based TF-IDF Precision@10: 0.274.

Additionally, a matrix factorization system was built using truncated SVD with k=10 latent factors over the repo-topic binary matrix. This system achieved Precision@10: 0.680, the strongest result across all systems.

The visual comparison will confirm that the topic-based SVD system outperforms both the hybrid content system and the baseline by a significant margin. SVD reaches Precision@10 of 0.680 because repository topics are explicit labels assigned by authors, a more reliable signal than free text descriptions or numeric features. The baseline fails because the most popular repositories belong to different technical areas and popularity does not imply topical similarity.

4.3 Offline evaluation report

Offline evaluation measures system quality without requiring real users. Three standard recommendation metrics are used, all computed at K=10. We will ask for the top 10 recommendations and then I will analyze what are the best.

- Precision@10 measures what fraction of the 10 recommended repositories share at least 2 topics with the query repository. **(are the recommendations topically relevant?)**
- Recall@10 measures what fraction of all relevant repositories in the catalog appear in the top 10. **how much of the relevant catalog does the system surface?**
- NDCG@10 (Normalized Discounted Cumulative Gain) measures ranking quality, a relevant repository appearing at position 1 is worth more than one appearing at position 10. **are the most relevant repositories ranked first?**

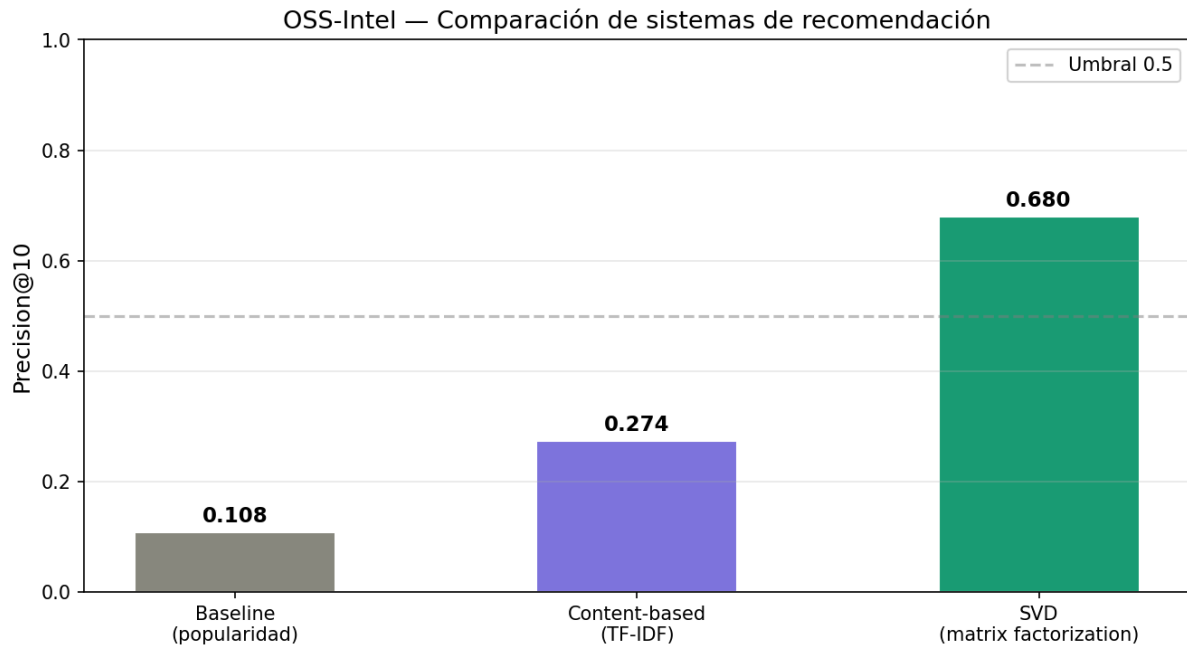
Relevance is defined as sharing at least 2 topics with the query repository. This threshold was chosen to avoid inflation from generic topics such as python or deep-learning that connect repositories from completely different domains. The candidate pool is the full catalog of 3,034 repositories excluding the query repository itself.

Evaluation A: Baseline vs Content-based (TF-IDF) vs SVD

The evaluation was run over a random sample of 50 query repositories with seed 42 for reproducibility.

En pocas palabras veremos si es relevante si al menos el repo que nos ha recomendando comparte 2 topics o mas con el repo que consultamos.

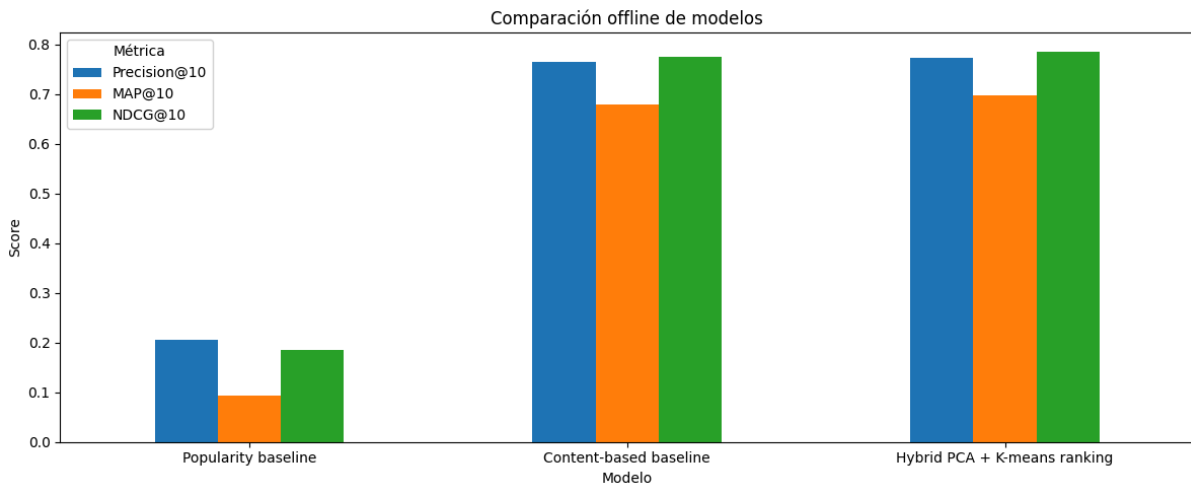
This evaluation uses a sample of 50 query repositories (seed 42) and a fixed relevance threshold (≥ 2 shared topics out of the 30 most frequent topics). It compares the popularity baseline, a pure content-based factorization (TF-IDF on the description), and the matrix factorization system SVD (k=10) on the repo-topic matrix.



Under this protocol, all metrics confirm the same hierarchy: SVD (0.680) outperforms TF-IDF (0.274), which in turn outperforms the baseline (0.108). The NDCG result is particularly relevant because it confirms that SVD not only recommends relevant repositories but also ranks them first. The baseline shows the lowest performance in NDCG, confirming that popular repositories are not only irrelevant but also poorly ordered with respect to the query.

Evaluation B: Baseline vs Content-based vs Hybrid PCA + K-means

This evaluation uses a larger sample (up to 500 query repositories) and a dynamic relevance threshold: sharing at least two significant topics is required if the query has three or more topics within the top 100, or at least one if it has fewer. The popularity baseline, a content-based baseline (TF-IDF on description + topics $\times$ 3 + language), and the hybrid system described in 4.2 (TF-IDF + PCA similarity + K-means cluster bonus + popularity + activity + growth) are compared.



Under this protocol, the popularity baseline performs the worst in all three metrics (Precision@10 $\approx$ 0.21, MAP@10 $\approx$ 0.09, NDCG@10 $\approx$ 0.18). The content-based baseline improves substantially (Precision@10 $\approx$ 0.76, NDCG@10 $\approx$ 0.78), and the hybrid system slightly surpasses it in all three metrics (Precision@10 $\approx$ 0.77, MAP@10 $\approx$ 0.70, NDCG@10 $\approx$ 0.79), indicating that PCA signals and the cluster bonus provide a marginal but consistent improvement over the textual content.

All three metrics confirm the same hierarchy: SVD outperforms TF-IDF which outperforms the baseline across Precision, Recall, and NDCG. The NDCG result is particularly meaningful because it confirms that SVD not only recommends relevant repositories but ranks them first. The baseline shows the lowest NDCG, confirming that popular repositories are not only irrelevant but also poorly ordered relative to the query.

4.4 Error analysis

This section reviews two types of cases:

- Strong cases: queries where the hybrid system achieves high accuracy.
- Failure cases: queries where the hybrid system achieves low accuracy.

The goal is not only to report numbers, but also to interpret why the system works or fails in certain repositories.

Class concepts applied:

- Overspecialization: a content-based system can become overly focused on very similar repositories and lose diversity.
- Metadata sparsity: although there is no user-item matrix, there may be little content information if a repository has few topics or a limited description.
- Popularity as bias: very famous repositories may appear at the top even if they are not the most thematically relevant.

Strong cases – mejores consultas del sistema híbrido:

	query_repo	relevant_candidates	Precision@10	Recall@10	MAP@10	NDCG@10
0	keras-team/keras	900	1.0	0.011111	1.0	1.0
1	ageitgey/face_recognition	954	1.0	0.010482	1.0	1.0
5	JaidedAI/EasyOCR	832	1.0	0.012019	1.0	1.0
6	donnemartin/data-science-ipython-notebooks	679	1.0	0.014728	1.0	1.0
9	terryum/awesome-deep-learning-papers	458	1.0	0.021834	1.0	1.0

Failure cases – consultas con menor desempeño:

	query_repo	relevant_candidates	Precision@10	Recall@10	MAP@10	NDCG@10
78	EutropicAI/Final2x	127	0.0	0.0	0.0	0.0
296	bytedance/SALMONN	38	0.0	0.0	0.0	0.0
403	ruvnet/RuVector	7	0.0	0.0	0.0	0.0
419	yazdotai/graph-networks	17	0.0	0.0	0.0	0.0
460	fangvv/LassoADMM	533	0.0	0.0	0.0	0.0

Strong cases belong to broad, popular domains (deep learning, CV, data science) with 450–950 relevant repos in the catalog. The Top-10 is easily filled with relevant items, giving Precision@10 = NDCG@10 = 1.0. Recall@10 is near zero (~0.01–0.02) simply because 10 items can never cover hundreds of relevant ones, a structural limit, not a flaw.

Failure cases show two patterns: niche repos with very few relevant candidates (RuVector: 7, graph-networks: 17), where the system has little chance of hitting one of so few targets (metadata sparsity / cold start); and repos with a decent relevant pool (Final2x: 127, LassoADMM: 460) that still

get 0 precision, suggesting the PCA/cluster-bonus signals pulled the ranking toward structurally similar but topically unrelated repos (overspecialization toward the wrong neighborhood).

4.5 Task framing explanation

The project is framed as a recommendation/ranking system fed by prior segmentation, not as collaborative filtering or supervised prediction:

System type	Applies?	Justification
Recommendation	Yes	Output is a ranked list of candidate repos for a query repo.
Ranking	Yes	Candidates are ordered by score (TF-IDF, PCA, SVD, or hybrid).
Prediction	No (not main task)	No supervised target variable is being predicted.
Collaborative filtering	No	No user-repo interaction matrix (ratings, clicks, playlists) exists.
Matrix factorization (collaborative)	No	SVD is applied to a repo-topic matrix, not to behavioral interactions.
Segmentation feeding ranking	Yes	PCA space and K-means clusters (from Week 7) act as additional proximity signals in the hybrid score.

PCA and K-means are not the final deliverable, they are inputs that enrich the ranking. The final deliverable is a set of comparable rankings (popularity, TF-IDF, hybrid, SVD) evaluated under the same offline protocol.

4.6 Justification for the absence of collaborative filtering

- Classic collaborative filtering requires a user-item matrix with observed interactions (ratings, clicks, favorites). The OSS-Intel dataset contains only repository metadata (description, language, topics, stars, forks, activity). There is no "user" entity, so building a user-repo matrix would require synthetic data.
- Instead, a co-occurrence proxy was built: a binary repo-topic matrix ($3,034 \times 30$) using the 30 most frequent topics. It has a density of 8.91% comparable to a real interaction matrix with each repo averaging 2.67 active topics.
- Truncated SVD ($k=10$) was applied to this matrix, in the same spirit as Week 9 matrix factorization, but over a repo<->topic relationship instead of user <->item. The 10 latent factors capture 58.42% of the variance, with the first component (12.21%) representing the dominant deep-learning/machine-learning axis of the catalog.⁷

This is methodologically honest: it applies the required factorization technique to the only matrix structure available (repo-topic), without claiming behavioral collaborative filtering.

4.7 Defense questions

1. Why no traditional collaborative filtering?

The dataset has no users or interactions. A repo-topic co-occurrence matrix was built instead, and SVD was applied to it. Fulfilling the matrix factorization requirement without inventing data.

2. Why does SVD (0.680) beat TF-IDF (0.274) and the baseline (0.108)?

Topics are explicit, author-curated labels. A cleaner thematic signal than free-text descriptions. Popularity has no relation to topical similarity at all.

3. Why a relevance threshold of " ≥ 2 shared topics"?

To avoid generic topics like "python" or "deep-learning" inflating precision artificially. Requiring 2 shared topics forces overlap on a more specific axis.

4. How were the hybrid weights (0.35/0.20/0.15/0.12/0.10/0.08) chosen?

Content (TF-IDF) is prioritized as the most interpretable signal, followed by PCA structural proximity and the cluster bonus (both inherited from Week 7's clustering). Popularity, activity and growth remain secondary so the system doesn't ignore project relevance/recency.

5. What role do PCA/K-means play if SVD performs best?

SVD and the hybrid system are two separate approaches: SVD is a pure topic-based factorization (4.1), while the hybrid system (4.2) tests whether segmentation signals (PCA, clusters) improve a content-based ranking. The hybrid does improve marginally over content-only (4.3), showing segmentation adds value even if SVD alone is stronger on topics.

6. What about cold start (a repo with no topics)?

Not handled explicitly. The current dataset has no such cases. In that scenario, TF-IDF/SVD scores would collapse, leaving PCA similarity and popularity as the only usable signals, a recognized limitation.

7. Does the offline evaluation reflect real user satisfaction?

No. Relevance is a topic-overlap proxy, not real clicks/ratings. The system measures whether the ranking surfaces topically coherent repos under a reproducible rule, not real developer preference.

Week 12(en verdad semana 13)

Análisis de Grafos y Reporte de Centralidad

El análisis de grafos transforma la representación del ecosistema de IA. En las semanas anteriores los repositorios eran filas en una tabla o vectores en un espacio de features. Esta semana los repositorios se convierten en nodos de una red donde las conexiones revelan estructura temática que las métricas de popularidad no pueden capturar.

La pregunta central de esta entrega es: ¿cuáles son los repositorios más influyentes en el ecosistema de IA científica más allá de su popularidad medida en stars?

5.1 Graph Definition

- **Nodos:** cada repositorio de GitHub es un nodo. El grafo tiene 1,938 nodos tras el filtrado.
- **Aristas:** dos repositorios están conectados si comparten al menos 4 topics declarados por sus autores. La arista significa co-ocurrencia temática, estos repositorios fueron etiquetados con los mismos topics. No significa que un repositorio dependa del otro, que sean funcionalmente idénticos, ni que uno causó la creación del otro.
- **Peso:** el peso de cada arista es el número de topics compartidos entre los dos repositorios. Mayor peso indica mayor similitud temática explícita.
- **Direccionalidad:** el grafo es no dirigido porque la co-ocurrencia de topics es simétrica, si el repo A comparte topics con el repo B, entonces el repo B comparte los mismos topics con el repo A.
- **Decisión de filtrado:** se probaron umbrales de peso entre 2 y 8. Con peso ≥ 2 el grafo tiene 362,471 aristas y densidad 0.082, demasiado denso, topics genéricos como python conectan todo con todo. Con peso ≥ 4 el grafo tiene 24,839 aristas y densidad 0.013, balance entre conectividad real y significado temático. Este umbral es una decisión de modelado que afecta a todos los resultados posteriores.

5.2 Graph Construction Script

El grafo se construye de forma reproducible desde el archivo repos_clean.csv ejecutando el notebook notebooks/Sem12_graph_analytics.ipynb en orden secuencial.

El proceso de construcción tiene cinco pasos:

- Paso 1: Cargar el dataset y parsear los topics de cada repositorio desde formato string a lista de Python.
- Paso 2: Para cada par de repositorios calcular el número de topics compartidos. Si dos repositorios comparten al menos 4 topics se crea una arista con ese peso.
- Paso 3: Construir el grafo con NetworkX agregando todos los nodos y aristas filtradas.
- Paso 4: Eliminar nodos aislados, repositorios que no comparten 4 o más topics con ningún otro repositorio. Se eliminaron 1,096 nodos aislados.
- Paso 5: Verificar la construcción: nodos, aristas, densidad y self-loops.

Resultado del script:

Métrica	Valor
---------	-------

Nodos totales	1,938
Aristas totales	24,839
Densidad	0.0132
Self-loops	0
Nodos aislados eliminados	1,096
Componentes conectados	34
Componente principal	1,861 nodos

Densidad mide qué fracción de todas las conexiones posibles realmente existen en el grafo. Fórmula simple: densidad = aristas existentes / aristas posibles

5.3 Graph Report

El análisis estructural se realiza sobre el componente principal de 1,861 nodos que representa el 96% de los repositorios conectados. Los 33 componentes restantes tienen entre 3 y 5 nodos cada uno, grupos muy pequeños de repositorios con topics muy específicos que no conectan con el ecosistema principal.

Componente	Nodos
Principal	1,861
Componente 2	5
Componentes 3-34	3 cada uno

5.3.1 Degree y Weighted Degree:

- El degree no ponderado cuenta cuántos repositorios vecinos directos tiene cada repositorio.
- El weighted degree suma los pesos de todas sus aristas, mide la fuerza total de co-ocurrencia temática acumulada.

Repositorio	Degree	Weighted Degree	Stars
d2l-ai/d2l-en	280	1,331	28,729
rasbt/LLMs-from-scratch	289	1,327	91,729
bharathgs/Awesome-pytorch-list	277	1,269	16,486

explosion/thinc	267	1,214	2,891
huggingface/datasets	265	1,198	21,469
lukasmasuch/best-of-ml-python	253	1,195	23,453
tangyudi/Ai-Learn	249	1,180	12,946
tensorzero/tensorzero	250	1,176	11,289
roboflow/supervision	243	1,090	38,259
explosion/spaCy	234	1,076	33,528

```
tensorflow weighted degree: 385
tensorflow degree: 89
```

(lo usaremos despues)

5.3.2 PageRank

- PageRank mide la importancia recursiva de cada nodo, un repositorio es influyente si otros repositorios influyentes están conectados a él. Se calculó con factor de amortiguación $\alpha=0.85$ sobre el componente principal. (alpha para entenderlo facil digamos que es un navegante que va por los nodos y artistas. entonces el 85% sigue una arista real y un 15% salta de un repo en repo sin pasar sin usar las artistas)

Repositorio	PageRank	Stars	Rank por Stars
rasbt/LLMs-from-scratch	0.00507	91,729	7
tensorzero/tensorzero	0.00434	11,289	58
d2l-ai/d2l-en	0.00404	28,729	25
bharathgs/Awesome-pytorch-list	0.00403	16,486	41
huggingface/datasets	0.00390	21,469	35
explosion/thinc	0.00373	2,891	127
roboflow/supervision	0.00365	38,259	17
lukasmasuch/best-of-ml-python	0.00360	23,453	30
tangyudi/Ai-Learn	0.00355	12,946	49
catalyst-team/catalyst	0.00354	3,376	117

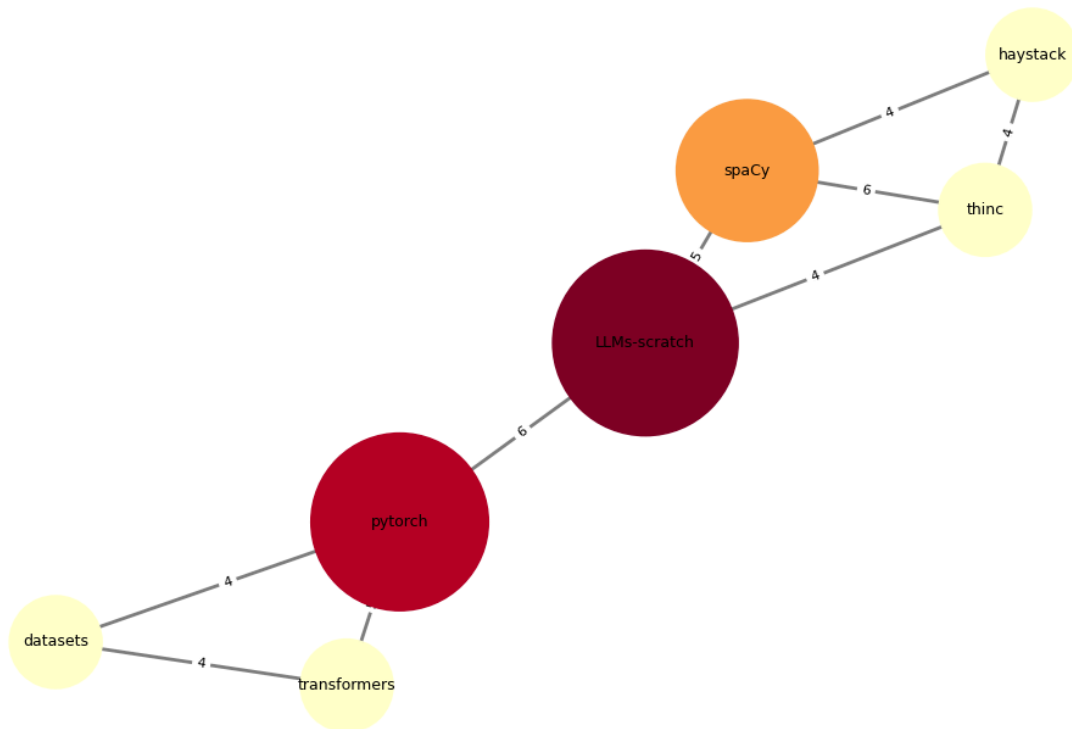
El valor de PageRank de 0.00507 significa que si tuviera a 10 000 navegantes moviéndose aleatoriamente alrededor del grafo, aproximadamente 50 de ellos estarían en rasbt/LLMs-from-scratch.

5.3.3 Betweenness

- Betweenness mide cuántos caminos más cortos entre otros pares de nodos pasan por cada repositorio. Un nodo con alto betweenness es un bridge, conecta regiones del grafo que de otro modo estarían menos conectadas. Los pesos se convirtieron a distancias (distancia = $1/\text{peso}$) antes del cálculo porque NetworkX interpreta pesos como distancias por defecto.

Repositorio	Betweenness	Stars
rasbt/LLMs-from-scratch	0.054	91,729
rohitg00/ai-engineering-from-scratch	0.034	5,834
tensorzero/tensorzero	0.034	11,289
datadreamer-dev/DataDreamer	0.026	1,113
huggingface/datasets	0.020	21,469

Ejemplo Betweenness — tamaño del nodo = valor de betweenness
El nodo más grande es el bridge más importante



5.4 Comparison section

El ranking por grafo revela repositorios influyentes que la popularidad por stars oculta completamente.

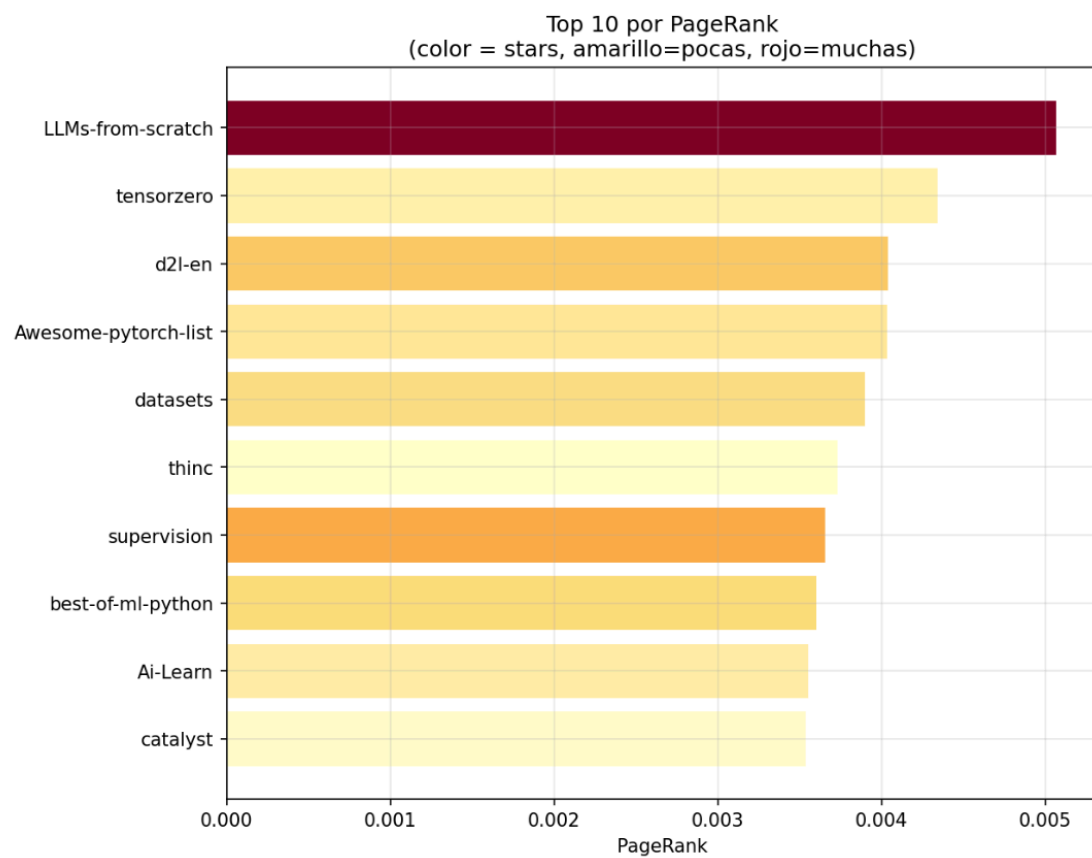
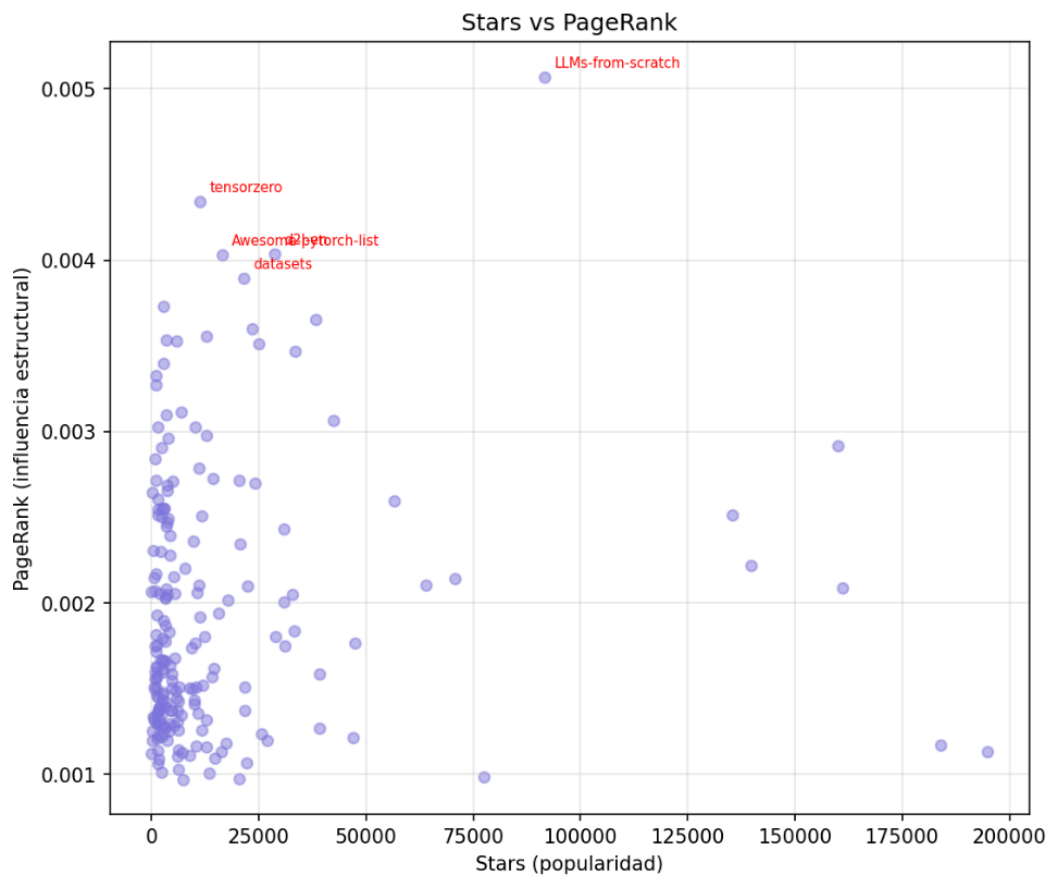
Repositorio	Rank PageRank	Rank Stars	Stars
rasbt/LLMs-from-scratch	1	7	91,729
tensorzero/tensorzero	2	58	11,289
explosion/thinc	6	127	2,891
catalyst-team/catalyst	10	117	3,376385
ayoolaolafenwa/PixelLib	15	180	1,072
tensorflow/tensorflow	—	1	194,936

COMPARACIÓN: PageRank vs Popularidad por Stars			
Repo	PR rank	Stars rank	Stars
rasbt/LLMs-from-scratch	1	7	91,729
tensorzero/tensorzero	2	58	11,289
d2l-ai/d2l-en	3	25	28,729
bharathgs/Awesome-pytorch-list	4	41	16,486
huggingface/datasets	5	35	21,469
explosion/thinc	6	127	2,891
roboflow/supervision	7	17	38,259
lukasmasuch/best-of-ml-python	8	30	23,453
tangyudi/Ai-Learn	9	49	12,946
catalyst-team/catalyst	10	117	3,376
rohitg00/ai-engineering-from-scratch	11	88	5,834
deepset-ai/haystack	12	28	25,024
explosion/spaCy	13	18	33,528
teddylee777/machine-learning	14	131	2,835
ayoolaolafenwa/PixelLib	15	180	1,072

Los hallazgos más importantes de esta comparación son tres.

- Primero, tensorflow/tensorflow tiene 194,936 stars, el más popular del dataset, pero no aparece en el top 15 de PageRank. Su weighted degree es solo 385 porque usa topics muy específicos de tensorflow que no comparte con muchos otros repos. Es famoso pero temáticamente aislado en el grafo.
- Segundo, explosion/thinc tiene solo 2,891 stars, rank 127 por popularidad, pero es rank 6 en PageRank. Es una librería técnica de NLP que conecta múltiples áreas temáticas del ecosistema. Completamente invisible por popularidad pero estructuralmente central.
- Tercero, ayoolaolafenwa/PixelLib tiene solo 1,072 stars, rank 180 por popularidad , pero aparece en el top 15 de PageRank. Es un repo de visión computacional que actúa como bridge entre el ecosistema de computer vision y machine learning general.

5.5 Interpretation note



5.5 Interpretation note:

- El grafo de co-ocurrencia de topics responde una pregunta que las semanas anteriores no podían responder: ¿qué repositorios conectan estructuralmente el ecosistema de IA científica?
- Qué significa una arista en este dominio: dos repositorios están conectados porque sus autores eligieron los mismos topics para describirlos. Esto es evidencia de similitud temática declarada explícitamente, no similitud de código, no similitud de funcionalidad, no dependencia técnica.
- Qué significa PageRank en este dominio: un repositorio con alto PageRank es visitado frecuentemente por un navegante aleatorio que sigue conexiones temáticas. En términos de negocio significa que ese repositorio es un hub temático, conecta muchas áreas del ecosistema y aparece naturalmente en el camino entre repositorios de distintas áreas.
- Qué NO significa la centralidad: alto PageRank no significa mejor código, mayor utilidad para un desarrollador, ni mayor actividad de mantenimiento. `explosion/thinc` tiene PageRank rank 6 con solo 2,891 stars, es influyente estructuralmente pero no necesariamente el mejor framework para un proyecto nuevo.
- Qué significa betweenness en este dominio: un repo con alto betweenness conecta áreas temáticas que de otro modo estarían menos conectadas. `rohitg00/ai-engineering-from-scratch` con solo 5,834 stars tiene betweenness de 0.034, actúa como puente entre el ecosistema LLM y el ecosistema de ingeniería de software aplicada.

Limitaciones del análisis:

- Los resultados dependen completamente de los topics declarados por los autores. Repos sin topics o con topics genéricos quedan subrepresentados.
- El umbral de peso ≥ 4 es una decisión de modelado. Un umbral distinto produce rankings distintos, demostrado en la tabla de sensibilidad al umbral.
- El análisis se realizó sobre el componente principal de 1,861 nodos. Los 1,096 repos aislados quedan fuera del análisis estructural.
- PageRank y betweenness se calcularon sobre un subgrafo de 200 nodos por eficiencia computacional, los resultados son representativos pero no exhaustivos.

5.6 Validity Checks

Se realizaron cuatro verificaciones de validez sobre el grafo construido.

- Verificación 1.- Simetría de la matriz de adyacencia:
La matriz A es simétrica ($A[i][j] = A[j][i]$) confirmado con `np.allclose(A, A.T) = True`. Esto valida que el grafo es correctamente no dirigido.
- Verificación 2.- Matrix de transición row-stochastic:
Todas las filas de P suman exactamente 1.0. Suma mínima: 1.000000. Suma máxima: 1.000000. Esto confirma que P es una matriz de transición válida para random walk.
- Verificación 3.- Self-loops: El grafo tiene 0 self-loops, ningún repositorio tiene arista consigo mismo. Correcto por construcción.
- Verificación 4.- Sensibilidad al umbral de filtrado:

Umbral	Aristas	Nodos	Componentes	Densidad
2	362,471	2,969	2	0.082
3	94,055	2,582	12	0.028
4	24,839	1,938	34	0.013
5	6,739	1,263	68	0.008
6	1,973	770	77	0.007
8	267	265	69	0.008

El umbral elegido de peso ≥ 4 es el punto donde el componente principal sigue siendo grande y las conexiones son temáticamente significativas. Umbrales menores conectan repos por topics genéricos como python. Umbrales mayores fragmentan el grafo en muchos componentes pequeños perdiendo estructura global.

5.7 Defense Questions

¿Qué representa un nodo y qué representa una arista?

Un nodo es un repositorio de GitHub. Una arista conecta dos repositorios que comparten al menos 4 topics declarados por sus autores. La arista es evidencia de similitud temática explícita, no dependencia técnica ni similitud de código.

¿Por qué el grafo es ponderado?

Porque no todas las conexiones temáticas tienen la misma fuerza. Dos repos que comparten 8 topics son más similares temáticamente que dos que comparten 4. El peso captura esa diferencia de intensidad.

¿Por qué el grafo es no dirigido?

Porque la co-ocurrencia de topics es simétrica. Si el repo A comparte topics con el repo B, entonces B comparte los mismos topics con A. No hay dirección causal en la relación.

¿Qué significa PageRank en este dominio?

Mide qué repositorios son visitados más frecuentemente por un navegante aleatorio que sigue conexiones temáticas. Un repo con alto PageRank es un hub temático que conecta muchas áreas del ecosistema de IA.

¿Cómo difiere el ranking del grafo del ranking por popularidad?

El ranking por popularidad mide stars acumuladas, visibilidad histórica. El ranking por grafo mide conectividad estructural, cuántas áreas temáticas conecta el repo. tensorflow tiene 194,936 stars pero PageRank bajo porque sus topics son muy específicos. explosion/thinc tiene 2,891 stars pero PageRank alto porque conecta NLP, MLOps y deep learning.

¿Cuáles son las limitaciones más importantes?

Primera: los resultados dependen de los topics declarados por los autores, una señal subjetiva y no estandarizada. Segunda: el umbral de peso ≥ 4 es una decisión de modelado que afecta todos los rankings. Tercera: betweenness y PageRank se calcularon

sobre subgrafo de 200 nodos, representativo pero no exhaustivo del componente principal completo.

11. Bibliography:

GitHub Staff. *Octoverse 2024: AI leads Python to top language as the number of global developers surges*. GitHub Blog, 2024.